
eve_rl Deep Review Consolidated Report

DualDeviceNav endovascular-navigation RL stack — four specialist review reports and the final synthesis

Compiled 6 July 2026

Sources: 1.md – 5.md (d:\Arjun)

Project: workspace/neve — eve_rl SAC/AWAC training stack

Contents

Part 1 — Documentation-History Analysis: eve_rl DualDeviceNav RL Project Docs evolution timeline

analysis · 1.md

- A. Evolution timeline
- B. Design-decision inventory (currently in force)
- C. Contradictions & smells
- D. Unresolved problems

Part 2 — Deep correctness review — eve_rl SAC/AWAC + PER stack Core RL algorithm bug hunt · 2.md

- A. Bellman target / done-flag flow (checklist 1)
 - B. AWAC actor loss (checklist 2)
 - C. log_std_min / log_std_max caps (checklist 3)
 - D. PER (checklist 4)
 - E. Target networks (checklist 5)
 - F. Entropy temperature α (checklist 6)
 - G. balanced_fraction / demo mixing (checklist 7)
 - H. Multi-worker / staleness / seeds (checklist 9)
 - I. Checkpoint save/load (checklist 10)
 - J. Dead code, wasted compute (checklist 11)
 - K. Eval mode (checklist 12)
- Priority summary

Part 3 — Training harness/config review Training harness/config review · 3.md

Review: LCCA AWAC v2 training orchestration (active run)

Part 4 — Env/reward/observation review Env/reward/observation review · 4.md

- 1. Reward correctness
 - 2. Projection / geometry
 - 3. is_on_correct_path / branch classification
 - 4. Observation (LocalGuidance, 30-dim)
 - 5. Termination / truncation
 - 6. Action interface
 - 7. Efficiency (summary of hot-path costs)
 - 8. Coordinate consistency — checked correct
- Obs/reward design gaps that plausibly cap LCCA-siphon success at ~50-60% (reasoned from code)

Part 5 — Deep Review: eve_rl Endovascular Navigation Stack Final synthesis (main session) · 5.md

- 1. The verdict
- 2. Bugs — Tier 1: shaping what the live run is learning right now
- 3. Bugs — Tier 2: the AWAC learning machinery
- 4. Bugs — Tier 3: environment internals and eval integrity
- 5. Documentation and process findings
- 6. Are you asking the right questions?
- 7. Overlooked and alternative strategies — the "next level" roadmap
- 8. What I'd actually do, in order

Documentation-History Analysis: eve_rl DualDeviceNav RL Project

Agent: Docs evolution timeline analysis · Source file: 1.md

Sources read in full: `RL_TRAINING_LIFECYCLE.md`, `RL_Improv_1.md`, `RL_Improv_2_analysis.md`, `RL_Improv_2_cursor_code_edits_analysis.md`, `RL_Improv_3.md`, `RL_IMPROV_4_CHANGES.md`, `RL_IMPROV_7_CHANGES.md`, `RL_IMPROV_8_CHANGES.md`, `RL_IMPROV_9_CHANGES.md`, `RL_IMPROV_10_CHANGES.md`, `HEURISTIC_FAILURE_FIX_IMPLEMENTATION_SPEC.md`, `HEURISTIC_SEEDING_TARGET_DIVERSITY_FIX.md`, `reward_analysis_problem.md`, `reward_analysis_solution.md`, `WAYPOINT_REWARDS_ANALYSIS.md`, `COMPREHENSIVE_TRAINING_ANALYSIS.md`, `HANDOFF_LCCA_V2_RELAUNCH.md` (all under `D:\Arjun\workspace\neve\`).

Note on chronology: docs #13–15 in the reading list (`reward_analysis_*`, `WAYPOINT_REWARDS_ANALYSIS`, `COMPREHENSIVE_TRAINING_ANALYSIS`) actually describe the **earliest** era (env1–env3, Feb 2026); I place them first below. Also: "Plan v12" (multi-target heatup harvest, IQL scaffold — per git log `662d5d4`) has **no dedicated changes doc** among these files; the trail after Plan v10 is only in `HANDOFF_LCCA_V2_RELAUNCH.md` and launcher comments.

A. Evolution timeline

Phase 0 — env1–env3 collapse era (Feb 2026). `COMPREHENSIVE_TRAINING_ANALYSIS.md` (run `diag_debug_test4`): SAC on the original waypoint env collapsed — insertion 743mm→221mm (−70%), alpha 0.61→0.0001, entropy +2.46→−4.67, Q peak 35→5.8; episode reward stayed flat ~−4.4, masking the collapse. `WAYPOINT_REWARDS_ANALYSIS.md`: env2 (10mm waypoints, 0.1 increment) had too-weak gradient (net +0.03 at 400mm → stop at ~47mm); env3 (5mm, 1.0 increment) created a local-optimum trap at 30mm (+0.14 safe vs +1.83 risky). Verdict: "reward structure needs fundamental redesign." `reward_analysis_problem/solution.md`: the `diagnose_collapse` tooling couldn't localize *where* the wire got stuck; proposed `stuck_run` detection (`cmd_translation`>0.05, $|\Delta\text{ins}| < 0.2\text{mm}$, ≥ 25 steps), executed-vs-commanded action logging, reward decomposition logging. **Left open:** reward decomposition logging and masked-action logging are never confirmed implemented anywhere later.

Phase 1 — env4 redesign (RL_Improv_1 + RL_TRAINING_LIFECYCLE). Intervention: continuous **arc-length progress reward** ($0.01 \times \Delta d_{\text{rem}} - 0.001 \times \text{cross_track}$, projecting only onto the correct-path polyline — no branch flipping), 8-dim **LocalGuidance** obs replacing 154-dim Centerlines2D, step penalty −0.005→−0.001, 3-stage **action curriculum** (catheter locked → ×0.1 → full), **heuristic seeding** of the episode-replay buffer with a centerline-follower (actions inverse-normalized to [−1,1]). `RL_TRAINING_LIFECYCLE.md` documents the multi-process architecture (16 workers, 1 GPU trainer, `VanillaEpisodeShared` 10k-episode buffer, batch 32 episodes, LSTM, `UPDATE_PER_EXPLORE_STEP=1/20`, eval every 250k on 98 fixed seeds) and the key insight that **500k heatup failed because the update-budget formula counts heatup steps** → 27,500 front-loaded gradient updates on random data hardened the nets. **Left open:** the formula itself was not changed (it bites again in RL_IMPROV_9 §15); heuristic translation constant-vs-proportional was flip-flopped 4× with the user (RL_Improv_1 §4).

Phase 2 — audits & infra (RL_Improv_2_analysis + cursor_code_edits, ~2026-03-27). No training yet; recommendations: normalize the 200:1-scale LocalGuidance features (§1.1), shrink the 11M-param `4×900 MLP + 500-LSTM` net to 2×256 (§1.2), raise update ratio (§1.3), **drop LSTM → step replay → PER/HER** (§1.4, §4.2–4.4), fix aggressive heatup ranges (§1.5), per-worker curriculum counter (§1.6), missing catheter reward (§1.7), PBRS reformulation of the lateral penalty (§4.1). Cursor edits doc: multiprocessing pickle fixes (`__reduce__`, `_eve_skip_config`), **parallel heuristic seeding** via `HeuristicActionFunctionFactory`, episode-arrival timestamping, replay-buffer persistence and resume. **Left open:** PBRS, catheter reward, feature ablation — never implemented in any later doc.

Phase 3 — RL_Improv_3. Implemented: feature normalization; **asymmetric action space** `gw/cath_trans ∈ [-10,+30]mm`, `rot ∈ ±1.5rad` (from evidence that 91.7% of heatup steps exceeded SOFA-safe thresholds); **target-diversity fix** (`HEURISTIC_SEEDING_TARGET_DIVERSITY_FIX.md`: deep-copied workers cloned `CenterlineRandom._rng` → 100 episodes had only 7 unique targets; fix = per-episode (`seed`, `{"target_branch"}`) schedule, round-robin branch-balanced); **min-success-rate seeding** (batched retry until ≥30% successes, capped success ratio ≤70%, cap $5 \times N$ episodes). **Left open:** items §5–14 (network size, update ratio, LSTM drop, step replay, PER, HER, curriculum counter, catheter reward, PBRS, ablation) explicitly PENDING.

Phase 4 — heuristic data quality (HEURISTIC_FAILURE_FIX spec → RL_IMPROV_4). Evidence: 116/320 seeded episodes were both-devices-maxed stalls (28.5% of compute), 71 positive-reward failures poisoning the buffer. Spec: heuristic-only detectors D1–D4, `TargetReached 1.0→3.0`, `FAILURE_TRUNCATION_PENALTY -5` on aborts + vessel-end (not max_steps), `OFF_BRANCH_GRACE_STEPS=10`. RL_IMPROV_4 then went further: **true branch membership** via KD-tree replacing the 5mm cross-track proxy (proxy had "100% false-negative rate at bifurcations"), LocalGuidance 8→14 dims (wrong/correct entry distance+direction), **wire-fold detector D5** (15 steps of insertion-without-arclength-progress), **wrong-branch shaped rewards** (–1 entry, –0.1/step), **unified –5 truncation penalty, heuristic/RL parity** (D1/D2/D4 deleted; both modes share detectors), always-retract `[-10,-1]` off-branch heuristic (cut wrong-branch aborts 59%→0%). Claimed validation: `test_on_branch.py` 99.6–99.8% correct on static points. **Left open:** P1 oscillation counter, P2 grace 20→50, P3 degenerate entry points, P4 wider tracking, P5 step replay, P6 SOFA checkpointing, P7 look-ahead rotation.

Phase 5 — RL_IMPROV_7 (Apr 2026): SOFA restore + heuristic controller wars. Wider tracking 5pts×10mm, Memory dropped (§1). SOFA checkpoint **collection** at 370–385mm (144 captures, §2) and **restore** — which was silently a no-op because `start.reset()` → `reset_devices()` zeroed everything three lines later; fixed by restoring after `super().reset()` (§7 Fix 5). Found **KD-tree branch flip-flop** at junctions (§3): `off_br` counter reset endlessly, 242/539 off-branch steps had net-positive reward (progress outran the –0.1 penalty). Fixes: interior points 15mm into branches (§4), **hysteresis debounce** (flip threshold 5, §7 Fix 2), `OFF_BRANCH_GRACE_STEPS 20→50` (§7 Fix 3 — required *because* the debounce removed the accidental reprieve). Heuristic rewritten: new `heading_err = angle(device_dir, tangent)` (old J-tip formula returned "garbage in the trunk", 47%-improve/52%-worsen noise, §10), two-phase retract/advance with threshold 1.2→2.0, three-regime Phase 2 (§11), **+1 CORRECT_ENTRY_REWARD** on daughter commit `arc_past ≥ 10mm` (§12, with step-1 pre-population fix §15.7), arclength `d_corr` migration (§9). Run-28 forensics (§14–15): **LVA-only success bias across all runs** (successes: 15/380, 28/302-reach, 2/174 — all LVA), dominant **aortic-arch wedge at (80,55,395)** for LCCA/RCCA/RVA, `nearest_named` early-step bias lesson, fluoro→vessel transform mandatory. **Left open (§16–17):** move insertion point up (open question), arch wedge, LCCA-wall lock.

Phase 6 — RL_IMPROV_8 (May 2026): path-aware state machine + per-daughter heuristics. `-- insertion_z 345` high-insertion A/B (§1–3): 4/50 vs 2/174 successes, trunk-folds eliminated, **but arch wedge statistically unchanged → wire-history hypothesis FAILED; geometry dominates.** Deeper bugs found (§4):

`is_on_correct_branch` checks only branch identity (wedge 13mm off-path still "on-branch"), `d_corr_arc` is projection arclength not distance, junction topology collapsed. Fix arc: on-path masks + `is_on_correct_path()` (§5), daughter-only entry markers (§6), **graph-routed d_corr** via BP-Dijkstra (§7, sister-branch case 86.4mm vs misleading 30mm Euclidean). Uniform cross-track tolerance thrashed (6mm and 10mm both fail — trunk drift is 9.5mm, §12); after surgical reverts (§13), replaced by the **state machine**: `_current_branch_idx` committed by junction crossings with `COMMIT_HYSTERESIS_MM=10` + **radius-aware tolerance** `max(2, 1.5×local_radius)` (§14) — wedges halved (35→17) but 0/50 successes. Two follow-up state-machine bugs: on-path-mask check dropped (Bug-A, §18) and the "impossible 20mm leap" junction-crossing condition (§19, 90% of transitions missing). Per-daughter heuristic policies with arclength-windowed **Phase A/B/C** and closed-loop J-tip bend rotation (§17, §22, §30); iteration table (§22) shows dynamic targets and flipped signs were catastrophic; identical code gave 28/50 vs 43/50 reach → **RNG seeding added (§25)**. Mesh inspection (§26): the RVA/RCCA "junction" is a **merged 12mm cavity** at z=412–417. Phase C factorial grid C0–C8 (§28): Strategy 1 SOFA-restore failed (36% wrong-location), Strategy 2 (4500 episodes) → **0 successes on all variants**. Reward grace + `_heur_suppress_wrong_branch` (§29). 500-ep per-daughter results (§31): LVA 19.0%, RCCA 2.9%, LCCA 1.4% (but 91.7% reach), RVA 0.6%. §32–38 (analysis only): 14-item heuristic/reward/obs discrepancy audit, `path_extension_set` proposal, **per-daughter-first RL architecture recommendation** (§35), PER-yes/HER-conditional (§36), fuller SOFA restore design (§38). **Left open**: everything in §31–38, bif1 region, dynamic replanning.

Phase 7 — RL_IMPROV_9 (Plans v5–v8): making the MDP trainable. Reward unification:

ArcLengthProgress gated on `is_on_correct_path()`, off-path = symmetric per-step Δ off-arc (first cumulative version created a 35:1 asymmetry, fixed to ~1.5:1) (§1, §3 table: success $\approx +5.6$ vs worst ≈ -6). Daughter-commit ± 1 state-machine events replace the +1 proximity reward (§2); wrong-branch penalty cascade **deleted** (§3). Obs enriched 14→28 (arclength-to/past-daughter, phase one-hot, bend_hat, Markov-completing counters, branch categoricals) (§4); tracking 10pts×15mm + **Memory re-added** + InsertionLengths → 76-dim obs (§5); 18-item non-Markov audit, `_last_on_path_arclen` fixed at the root (Option C) (§6). RCCA-only scoping via `default_target_branch` (§11 — without it explore/eval would silently be 4-daughter). **Step-RL migration (Plan v6, §13)**: `VanillaStep` off-by-one dropped every terminal transition (fixed), buffer 1e6 transitions, batch 256, update ratio 1.0, **no-embedder MLP 256×256** (the `ff_only` branch has a latent bug, bypassed), lr 3e-4 (stale Optuna 2.2e-4 abandoned). **PER (Plan v7, §14)**: sum-tree, $\alpha=0.6$, β 0.4→1.0; **HER ruled out** (reward is path-structured + per-episode mutable state unreconstructable). Then four consecutive **critic divergences**: root causes (a) update-budget formula counting seeding as heatup → 163k front-loaded updates (§15, fixed: budget from exploration only) and (b) **MDP grounding** — failure truncations returned `terminated=False`, so ~99.6% of episode ends bootstrapped; fix: `if truncated: terminated=True` + retro-patched caches (§17). Plan v8 stabilization knob matrix (§16): `--grad_clip`, `--update_per_explore_step`, `--demo_priority_bonus` (DQfD floor), `--priority_mode {td,return,outcome}`, `--balanced_fraction` (two-stream clean sampler), `--algo awac (exp(A/λ)-weighted cloning)`.

Phase 8 — RL_IMPROV_10 (Plans v9–v10): RCCA restore-at-fork actually works. The **inversion bug hunt** (§11): RCCA-threading episodes scored worse than failures; three causes — lateral penalty -3 of unavoidable trunk drag (fixed by **radius-aware deadband**, §6), `_committed_forks` reset as `list` not `set` → 2544 swallowed `list.add` exceptions, zero +1 commits ever (§1), and -0.05 firing on RVA↔RCCA classifier flicker (fixed: fires only at ≥ 10 mm off-arc, latched, §3). Reward redesign: **+1 at every on-path junction** (max +3 RCCA), **path-segment-conditioned step penalty** (trunk -0.007→-0.002, bridge/daughter 0, off-path -0.007) replacing uniform -0.001 (§4), 2× progress in target daughter (§5). Standing rule minted: *never change the*

branch classifier to fix a reward (a debounce attempt dropped heuristic threading to 0% because the heuristic reads the same flag; reverted, §11.4). **Fuller SOFA restore** implemented (7 DOF fields, restore rotation, settle 50→3, §7); restore-at-fork pipeline (pre-bif11 pool of 148 checkpoints, single-run seed→heatup→10k-pretrain→online, §8). Verification (§12): critic stable through 85k updates (`q1_loss` 0.004–0.026, `grad_clip` 1.0 barely binds), reward ordering correct. **Plan v10**: `--log_std_min -2` entropy floor (§13a) + 5 curated good start states (§13f) + heuristic-from-restore seeding after **random heatup-until-N produced zero successes** (§13b — recorded finding: random exploration cannot seed fork-commit problems); eval also wrapped with restore (§13d), first-restore double-restore fix (§13e). `rcca_awac_v3` run (§14): strict 32.7/31.6/36.1% over 3 evals; **per-state spread 10%→82%**; entropy_proxy fell -2.3→-10.2 anyway — §15 decomposition shows `log_std_min` only floors the Gaussian term; the **tanh-Jacobian is unbounded as the mean rails** (clamp_fraction 0.06→0.53). Eval-only sweep from best state pid19116 (§16): **84/98 = 85.7% strict, 96.9% functional** — per-state difficulty, not policy quality, was the bottleneck. **Functional success** metric introduced (terminal `cum_reward` $\in [-1, 1]$, §17); anatomical mapping (§18): "RCCA" centerline runs 237mm to the carotid siphon; all remaining failures are catheter-overshoot folds in the siphon band (46.2% there vs 100% in proximal CCA). §19 decision tree: solved = critic stability, fork commit, per-state attribution; unsolved = saturation collapse, λ , per-state curriculum, siphon.

Phase 9 — LCCA from z=345 (HANDOFF_LCCA_V2_RELUNCH, Jul 2026). Same AWAC/PER/balanced stack applied to LCCA "Challenge-1" (no restore, full trunk). `lcca_awac_v1` soft-collapsed at ~7% eval; forensics found (1) a `log_std` **CEILING explosion** (+2 ceiling, std→2.0, actions railed by sampling noise — explicitly *not* the floor collapse first suspected) and (2) **structural LVA-confusion**: ~38% of episodes end in LVA even when entropy was healthy; successes perform a leftward "hook" at bif2 100% of the time, LVA failures 6%. v2 = EXP-1 `--log_std_max 0.0` (validated: entropy flat ~2.65 through 470k) + EXP-2 `--balanced_fraction 0.3→0.6`. First v2 run killed at 470k by host interruption with one eval (3.06%@257k, *below* stochastic rollouts 7–11%). Seed buffer `lcca_awac_seed_v1.npz`: 8,738 episodes / 1.07M transitions, `is_clean=1012` all-LCCA. **Left open**: the entire v2 question (does Quality beat 7%, does LVA% drop) and EXP-3 (fork obs feature / ostium waypoint reward — requires human approval).

B. Design-decision inventory (currently in force)

Reward (env5, post-Plan-v9; refs RL_IMPROV_9 §3 table + RL_IMPROV_10 §3–6): 1. **TargetReached +3** — justification: success must clearly dominate short failures (spec §7). Tested via reward-distribution separation; solid. 2. **ArcLengthProgress +0.01/mm, gated on `is_on_correct_path()`**; off-path = $-0.01 \times \Delta(\text{off-arc})$ symmetric delta (retraction earns positive) — justification: heuristic/reward agreement (RL_IMPROV_9 §1); the delta-vs-cumulative choice was corrected after an asymmetry calculation, not an experiment. **2× progress inside target daughter moving forward** (RL_IMPROV_10 §5) — replaces a dwell-rewarding constant; reasoning sound, untested in isolation. 3. **Lateral penalty $-0.001 \times \max(0, \text{cross_track} - \text{local_tolerance})$** (radius-aware deadband, RL_IMPROV_10 §6) — justification: 5mm trunk wall-riding is unavoidable, was -3/episode drag. Verified as part of the inversion fix. 4. **+1 per on-path junction commit, latched** (RL_IMPROV_10 §3) — justification: dense credit for each fork decision; accepted side effect: partial-fail nets +1.7 (positive). Verified only via reward-ordering (+3..+7 >> +1.7 >> +0.2). 5. **-0.05 on deep wrong-commit** ($\geq 10\text{mm}$ off-arc since divergence, latched per divergence) — tuned specifically to survive the RVA/RCCA tolerance-asymmetry flicker. Magnitude arbitrary. 6. **Path-segment step penalty** (trunk $-0.007 \rightarrow -0.002$ interpolated; (0) -0.002 ; bridge/daughter 0.0; off-path -0.007 ; RL_IMPROV_10 §4) — RCCA-topology-specific

(regex on branch names `\(2\)`, `\(0\)`, `\(11\)`); portability to other daughters unstated. 7.

FAILURE_TRUNCATION_PENALTY -5 on wbt/fold/vessel-end, not max_steps (spec §6, RL_IMPROV_4 §6) — rationale: early aborts remove future step penalties. Long-standing, never re-examined after the step penalty changed shape in §4 (max_steps episodes no longer accumulate ~-1 uniformly).

Termination/truncation semantics: 8. **All truncations set `terminated=True`** ("absorbing failure state", RL_IMPROV_9 §17) — the single most load-bearing fix; justified by batch evidence (0/13 failure ends grounded) and validated by subsequent critic stability. Note this makes truncation-vs-termination semantically meaningless downstream and bakes "failure is unrecoverable" into the value function. 9. Truncation triggers: `wrong_branch_timeout` at 50 consecutive off-path steps; fold-stall (insertion without arclength progress); vessel-end; sim-error; max_steps. Grace values (50, 15-20) empirically tuned per RL_IMPROV_7 Fix 3 and never revisited after the state machine replaced the debounce.

Observation (76-dim, RL_IMPROV_9 §5): 10. `Tracking2D(10, 15)` + `Memory(2, FILL)` (velocity is genuine hidden state — Memory was removed in IMPROV_4/7 then re-added), `target` 2D, `last_action` 4, `inserted_lengths` 2, `guidance` 28. Justified by the §32 audit (B1/B2/B3) + 18-item non-Markov audit; never ablated. 11. Guidance includes a **phase one-hot read from `_heur_rva_phase`** (RL_IMPROV_9 §4) — defined by the heuristic's presence; during pure RL exploration/eval it reads a getattr default. No doc examines what the policy sees for this feature at eval time. Weakly justified. 12. Branch state machine (radius-aware tolerance $K=1.5$, commit hysteresis 10mm, on-path masks) is simultaneously the **classifier for reward, heuristic gating, and observation features** — a deliberate unification, protected by the standing rule (RL_IMPROV_10 preamble).

Algorithm & replay: 13. **AWAC ($\lambda=3.0$)** over SAC — chosen because every SAC variant diverged; validated (critic stable 750k+ steps, RL_IMPROV_10 §14). $\lambda=3$ untuned; raising to 6-10 is an explicitly untried lever (§19.2). 14. **Step replay** (1e6-11M transitions, batch 256, update/explore = 1.0) + **PER** (α 0.6, β 0.4-1.0) + `--balanced_fraction` clean-lane sampler (0.6 for LCCA) + `demo_priority_bonus`. PER canonical values, never A/B'd against uniform despite the doc claiming the design permits clean attribution (RL_IMPROV_9 §14). 15. **grad_clip 1.0** — despite RL_IMPROV_9 §16 arguing for 10.0 ("1.0 would clip every healthy update too"); RL_IMPROV_10 §12 reports 1.0 "barely binds". No doc records the change decision. 16. **MLP 256x256, no embedder** (LSTM dropped; `ff_only` path avoided due to latent bug), lr 3e-4, 10k warm-start pretrain, update budget from exploration steps only (RL_IMPROV_9 §13, §15). 17. **Entropy handling:** `log_std_min -2` floor + `log_std_max 0.0` ceiling (HANDOFF EXP-1). Floor demonstrably insufficient (RL_IMPROV_10 §15); ceiling validated for the ceiling-explosion mode only. No entropy term in the AWAC loss — the actual fix (explicit entropy bonus / tanh-margin penalty) is still deferred.

Architecture & training strategy: 18. **Per-daughter policies** (not a shared model) — argued in RL_IMPROV_8 §35 (seeding imbalance 30x, distinct Phase-B/C steering, baselines needed); the shared-model + per-target buffers + PER/HER stack (§36) was never run. 19. **Heuristic warm-start pipeline:** per-daughter phase-based heuristic (arclength-windowed Phase A/B/C, closed-loop J-tip bend rotation, original sign convention — empirically validated in the v2-v8 grid, RL_IMPROV_8 §22), branch-balanced `(seed, options)` schedule, min-success-rate collection with $\leq 70\%$ success cap, `clean_thread` = final-branch equality (RL_IMPROV_10 §2), `is_demo` tagging. 20. **Start-state strategy:** RCCA = restore-at-fork from 5 curated SOFA checkpoints (fuller 7-field restore, settle 3); LCCA = z=345 no restore ("Challenge-1"). Explicitly two opposing regimes; per-state difficulty is known to dominate the metric (§16). 21. **Eval protocol:** deterministic mean-action, 98 fixed seeds, every 250k explore steps; "Quality" = strict TargetReached fraction; eval env wraps with the same restore wrapper as train (RL_IMPROV_10 §13d) — except in the LCCA campaign where there's no restore. "Functional success" ($RE[-1,1]$) exists as a secondary lens but Quality remains the headline. 22. **Standing rules**

(RL_IMPROV_10 §11, §19; HANDOFF §7): never modify the branch classifier to fix reward; one variable under study per iteration; reward/obs/terminal/dynamics changes require explicit human approval.

C. Contradictions & smells

Direct reversals of earlier fixes: 1. **Heuristic controller triple-reversal:** `HEURISTIC_FAILURE_FIX` spec §"Controller Changes" mandates removing the `max(gw_trans, 5.0)` floor, adding a device-length-aware cap, and adding a rotation cap near target. `RL_IMPROV_4` §11 does the exact opposite of all three (re-adds the 5mm floor, removes the rotation cap, removes the device-length cap) with a one-paragraph rationale. No experiment cited for either direction. 2. **Memory wrapper:** removed as unnecessary ("wire shape is sufficient", `RL_IMPROV_4` P4, `RL_IMPROV_7` §1) → re-added as necessary ("velocity is genuine hidden state", `RL_IMPROV_9` §5). Tracking config churned 3pts×2mm → 5×10 → 10×15 with a cache-invalidation cost each time. 3. **Hysteresis/debounce lineage:** debounce added (`IMPROV_7` Fix 2) → made obsolete by state machine ("no separate debouncer needed", `IMPROV_8` §14) → left as dead code (`IMPROV_9` §6 item 14) → a *new* debounce attempted and reverted (`IMPROV_10` §11.4) because it broke the heuristic. The same flicker problem was attacked five separate ways (proxy threshold, interior points, debounce, state machine + radius tolerance, reward-side latch). 4. **HER:** recommended repeatedly (`RL_Improv_2` §4.2, `RL_Improv_3` §10) and analyzed as "huge for LCCA — its 1.4% becomes effectively 80%+" (`RL_IMPROV_8` §36), then **categorically ruled out** (`RL_IMPROV_9` §14: reward is path-structured, state unreconstructable). The §36 analysis was written after the path-structured reward already existed — the contradiction is between two analyses of the same code, one of which is wrong. 5. **grad_clip 10.0 vs 1.0** (`RL_IMPROV_9` §16 vs `RL_IMPROV_10` §12/HANDOFF §5): the earlier doc explicitly rejects 1.0 with a quantitative argument; the working stack uses 1.0 with no recorded rationale for the switch. 6. `--insertion_z` / `--checkpoint_dir` **mutual exclusion** (`IMPROV_8` §1) later relaxed (`IMPROV_10` §8b) — fine, but the guard's original justification ("opposing strategies") was quietly invalidated. 7. **OFF_BRANCH_GRACE_STEPS 10 → 20 → 50:** each change compensated for a side effect of another fix (spec §3 says "recoveries happen in 2–7 steps, 10 covers those"; the debounce then made 20 lethal to slow recoveries; 50 chosen). The parameter encodes the history of other bugs rather than a principled quantity.

Fixes fighting fixes / whack-a-mole reward engineering: 8. Wrong-branch penalties: −1 entry + −0.1/step added (`IMPROV_4` §5) → 3-step grace bolted on for flicker (`IMPROV_8` §29) → **entire cascade deleted** as "heuristic-era hacks" (`IMPROV_9` §3). Correct-entry +1: proximity-gated (`IMPROV_7` §12) → daughter-fork-gated (`IMPROV_8` §6) → state-machine ±1 events (`IMPROV_9` §2) → +1 at every junction with a −0.05 deep-commit penalty (`IMPROV_10` §3). Five reward regimes in four docs for the same behavioral target. 9. Magnitude inconsistency inside that lineage: `IMPROV_9` §2 specifies **−1** for a wrong-daughter commit; `IMPROV_10` §3 refers to "the original **−0.05**" firing on flicker. Either an undocumented intermediate change or a doc error — no doc bridges it. 10. Fold-detector constants disagree between docs describing the same code: `FOLD_STALL_STEPS=15`, condition `Δgw≥0.5 ∧ Δs<0.5` (`IMPROV_4` §4) vs "`_fold_stall_count` incremented when `delta_gw/delta_s > 5` ... exceeds `FOLD_STALL_STEPS = 20`" (`IMPROV_7` §8). 11. The −5 truncation penalty's exclusion of max_steps was justified by "1000 steps of −0.001 already accumulates −1" (`IMPROV_4` §6); after `IMPROV_10` §4 the uniform −0.001 no longer exists (on-path bridge/daughter steps cost 0), so a max_steps dwell deep in the daughter can now end near 0 without the −5 — the original justification is stale and nobody re-checked it.

Claims without evidence / weak validation: 12. `IMPROV_4` §1 claims branch membership "validated... 99.6–99.8% correct" — the test was static points; the actual failure mode (per-step flip-flop at junctions under tip

jitter) was exactly what the static test could not catch, and it dominated the next two doc cycles (IMPROV_7 §3, IMPROV_8 §4). 13. Nearly every heuristic-policy conclusion before IMPROV_8 §24 was drawn from N=50 unseeded runs; §24 then demonstrates identical code produced 28/50 vs 43/50 (± 15 -episode noise). E.g. the v3 "Phase C caused 2/50" regression conclusion is explicitly retracted ("cross-run noise dominates"), but v4–v7 design decisions had already been made on the same noise floor. 14. IMPROV_8 §15 v5 run: 0/50 successes vs baseline 4 attributed to "sample variance ($P(0) \approx 1.5\%$)" — a probability that argues *against* the variance explanation, in the same bullet that proposes it. 15. RL_IMPROV_9 §10 verification plan Phases 1–2 (regression targets, success $\geq 5\%$) — no doc reports Phase 1 results; the narrative jumps straight to divergences. 16. IMPROV_10 §13f lists per-state eval#1 rates (70/41/29/17/10%) that disagree with the §14 table for the same eval (81.8/50/28.6/16.7/10%) — inconsistent denominators, unexplained. 17. IMPROV_8 §30 vs §31: preliminary per-daughter results (LCCA 4%, RVA 0.4%...) silently revised (LCCA 1.4%, RVA 0.6%...) in the next section.

Same problem recurring under different names: 18. **Do-nothing/shallow-insertion attractor**

(COMPREHENSIVE §5; env2/3 waypoint traps; IMPROV_7 §8 "first batch collapsed into do-nothing"; IMPROV_10 §13b random heatup can't commit forks) — each generation re-derives that sparse/misshaped reward + easy negative signal $\Rightarrow$ freeze. 19. **Entropy collapse** appears as three genuinely different mechanisms treated as one symptom: alpha auto-tune collapse under SAC (COMPREHENSIVE §6), AWAC mean-rail via unbounded tanh-Jacobian despite σ floor (IMPROV_10 §15), and the LCCA `log_std` **ceiling** explosion (HANDOFF §1, "NOT the floor collapse we first suspected"). Fixes for one were initially applied to another. 20. **Classifier flicker** (KD-tree flip-flop $\rightarrow$ tolerance thrash $\rightarrow$ state-machine flicker at asymmetric-tolerance forks $\rightarrow$ reward flicker inversion) is the single most persistent defect (IMPROV_7 §3, IMPROV_8 §12/§15/§29, IMPROV_10 §3/§11). 21. **Restore lands in the wrong place**: silent no-op restore (IMPROV_7 Fix 5), 36% wrong-location (IMPROV_8 §28), first-restore double-restore (IMPROV_10 §13e). Three separate root causes in the same subsystem. 22. **Critic divergence** (IMPROV_9 §15, §16, §17): four runs, three stacked root causes (update-budget spike, no gradient clipping, ungrounded bootstrap); early fixes were declared per-run ("the §15 fix") before the true cause (§17 grounding) was found.

Metric definition drift: 23. "Success" means, in different docs: positive total reward (IMPROV_7 §2, $\sim 2\%$); analyzer `SUCCESS = R > 1.0` (IMPROV_8 §9); `term=True` strict TargetReached (§30–31); "reach %" (touched cur_branch); `clean_thread` — itself redefined twice (latch-based, IMPROV_9 §7 $\rightarrow$ `reached_target_daughter` $\wedge$ `-wrong` §12 $\rightarrow$ final-branch equality, IMPROV_10 §2); **functional success** `RE[-1,1]` (IMPROV_10 §17 — a band that the IMPROV_8 analyzer would have labeled failure, and which sits *below* its old $R > 1.0$ success line); eval "Quality" (strict, 98 seeds). Cross-doc rate comparisons (e.g. "8% vs 1.1%", "19.6%") mix these definitions. 24. IMPROV_10 §17 itself shows strict success is **classifier noise** across evals (wfs $\leftrightarrow$ wbt flip made eval#2 look like a regression when functional says it improved) — i.e., the headline metric used for every go/no-go up to that point was partly measuring which truncation fires first. 25. Reward-scale drift makes cross-run reward comparisons meaningless: worst-case episode reward went ≈ -218 (IMPROV_4 P1) $\rightarrow -53$ thrash plateau (IMPROV_8 §12) $\rightarrow -6$ floor (IMPROV_9 §3); "positive reward" as a success proxy (IMPROV_7) is incompatible with the post-v9 regime where partial failures net +1.7 (IMPROV_10 §3, acknowledged).

Other smells: 26. LVA's (19) continuation is handled by a per-policy suppression flag (`_heur_suppress_wrong_branch`, IMPROV_8 §29–30) that the docs themselves call a hack, with the clean fix (`path_extension_set`, §33) designed in detail and then deliberately descoped (IMPROV_9 header) — the hack is still load-bearing. 27. The 76-dim observation contains heuristic-phase features that are only populated when a heuristic is driving (B3 fix); no doc addresses train/inference distribution shift for these features. 28. `RL_TRAINING_LIFECYCLE.md` (still the "system overview") describes the episode-LSTM stack, 10k-episode

buffer, 1/20 update ratio, and Optuna LR — all superseded by Plan v6+; the doc was never updated, so the entry-point document contradicts the current system. 29. Per-daughter policy files write to `_heur_rva_phase` "to fool the STEP-log writer" (IMPROV_8 §32 C4) — acknowledged cosmetic hack that also feeds observation feature plumbing (IMPROV_9 §4 reads the same field).

D. Unresolved problems

Explicitly admitted open: 1. **Action saturation / entropy collapse under AWAC** — `log_std_min` insufficient by construction; explicit entropy bonus in the AWAC loss and tanh-margin penalty deferred (RL_IMPROV_10 §15, §19.1). `log_std_max 0.0` (HANDOFF EXP-1) treats the LCCA ceiling mode only; the mean-rail mechanism is untreated. 2. **LCCA LVA-confusion (~38%, structural)** — the whole point of the pending v2 relaunch; EXP-3 (fork-disambiguating obs feature / ostium waypoint reward) specified but blocked on human approval (HANDOFF §1, §7). Run killed at 470k with one eval; question unanswered. 3. **Carotid-siphon ceiling** — 46.2% success in the z=575–601 band; catheter-overshoot fold mechanism; siphon-biased target sampling proposed, not done (RL_IMPROV_10 §18, §19.4; MEMORY: "siphon ceiling unsolved"). 4. **Per-state curriculum** — pid10145/pid20043 drag mixed-state numbers; narrowing proposed, not done (§19.3). 5. **awac_lambda 3→6–10** and **update_per_explore_step 1.0→0.5** — identified levers, untested (§19.2, §19.5). 6. **Aortic-arch wedge at (80,55,395)** — declared mechanical/geometric (wire-history hypothesis failed, IMPROV_8 §3); never solved, only sidestepped by restore-at-fork for RCCA; still the dominant failure for full-trunk starts. 7. **Phase C / daughter-threading heuristic depth** — 4500-episode factorial grid gave 0 successes on every variant; doc concludes "Phase-C steering choice isn't the bottleneck... the grid may not be the right next experiment" (IMPROV_8 §28, §38 status). Depth ceiling (arc≈26–43mm) unexplained for RVA/RCCA heuristics. 8. **path_extension_set unification** — designed (IMPROV_8 §33), only the RCCA subset of the audit was implemented; LVA still runs on the suppression hack (IMPROV_9 scope note). 9. **LCCA Phase-C variant dispatch (C1)** and other audit items B4/B5/D2 — deferred (IMPROV_8 §32/§34c, IMPROV_9 scope note). 10. **Fuller-restore verification target** — §38 set "wrong-location <10% (from 36%)" as the acceptance test; RL_IMPROV_10 §7 implements the fuller save but no doc reports the re-measured wrong-location rate. 11. **Shared-model experiment** (per-daughter buffers as init, PER + per-target balancing + one-hot daughter id) — the stated "interesting research question", never run (IMPROV_8 §35–36). 12. **Non-Markov residuals** — contact-force/LCP warm-start state (audit item 18) and fold-detector delta baselines (item 8) accepted as unresolved (IMPROV_9 §6). 13. **Bif1 region** (6 branches at z≈70) never investigated for femoral-entry configs (IMPROV_8 §11). 14. **Dynamic path replanning** (Dijkstra2 for recovery routes) deferred (IMPROV_8 §11). 15. **Deterministic-eval-below-stochastic-rollout gap** for LCCA (3% vs 7–11%) — flagged as the "yellow flag" pointing to EXP-3; unresolved (HANDOFF §1, §7).

Dropped silently (no later mention of resolution or explicit deferral): 16. **Catheter reward signal** (Improv_2 §1.7, Improv_3 §12 PENDING) — never built; ironically the terminal failure mode of the best policy (catheter-overshoot wfs) is exactly a catheter-control failure. 17. **PBRs reformulation of the lateral penalty** (Improv_2 §4.1, Improv_3 §13) — vanished; the lateral term evolved (deadband) but never into potential form. 18. **LocalGuidance feature-ablation study** (Improv_2 §2.2, Improv_3 §14) — never run; obs meanwhile grew 8→14→28 with zero ablation at any size. 19. **Curriculum per-worker step counter** (Improv_2 §1.6, Improv_3 §11) — never fixed; curriculum quietly abandoned (OFF for RCCA, IMPROV_9 §11) but its Stage-2 distortion of heuristic demos (audit E2) was left categorized "cosmetic". 20. **Wrong-branch oscillation entry counter (P1)** — "largely superseded" claim (IMPROV_7 §17) but the oscillation pattern reappears at the RCCA/RVA fork in IMPROV_10 §3; the entry-count idea never revisited. 21. **Reward-decomposition per-step logging and**

executed-vs-masked-action logging (reward_analysis_problem/solution) — the "best stuck locator" proposals; no later doc confirms implementation (diag went the snapshot/STEP-log route instead). 22. **Heuristic look-ahead rotation P7** (IMPROV_4) — superseded in spirit by Phase A/B/C but never evaluated as designed. 23. **Legacy method cleanup** (`is_on_correct_branch`, `_path_junction_arclengths`, dead hysteresis fields) — flagged for removal "once new metrics validated" (IMPROV_8 §11); never removed per later file indexes. 24. `--heatup_until_successes` **infrastructure** — built end-to-end, abandoned after zero successes, left in-tree unused (IMPROV_10 §13b). 25. **Plan v11/v12 documentation gap** — git log and HANDOFF reference "Plan v9–v12 history", multi-target heatup harvest, rolling save, and an **IQL scaffold**; none of the read docs describe v11/v12, so the current LCCA/LICA workstream's design rationale exists only in launcher comments and memory files. 26. **RL_TRAINING_LIFECYCLE.md staleness** — the architecture overview was never revised after the episode-LSTM → step-MLP migration; anyone onboarding from doc #1 learns the wrong system.

Deep correctness review — eve_rl SAC/AWAC + PER stack

Agent: Core RL algorithm bug hunt · Source file: 2.md

Scope actually reviewed (active path for `launch_lcca_awac_v2.sh`: `Synchron` agent, 16 CPU workers + 1 CUDA trainer, `PERVanillaStepShared` step buffer, `SAC(algo="awac")`, MLP 256×256, no LSTM embedder):

- `D:\Arjun\workspace\neve\eve_rl\eve_rl\algo\sac.py` (full), `algo\iql.py` (skim), `algo\algo.py`
- `model\sacmodel.py`, `network\gaussianpolicy.py`, `network\qnetwork.py`, `network\component\mlp.py`, `component\lstm.py`, `component\dummy.py`
- `replaybuffer\pervanillastep.py`, `vanillastep.py`, `vanillaepisode.py` (methods), `vanillashared.py`, `pervanillashared.py`, `replaybuffer.py`
- `agent\single.py`, `agent\synchron.py`, `agent\singleagentprocess.py`, `agent\agent.py`, `runner\runner.py`, `optim\optim.py`, `util\experience_cache.py` (skim)
- Consumers: `training_scripts\util\agent.py`, `training_scripts\DualDeviceNav_train.py` (relevant sections), `training_scripts\util\env5.py` (terminated/truncated), `launch_lcca_awac_v2.sh`

A. Bellman target / done-flag flow (checklist 1)

A1. CONCEPTUAL — `truncated → terminated=True` grounds *time-limit* truncations as absorbing states.

`training_scripts\util\env5.py:1422-1423` rewrites `terminated=True` for every truncation (wrong-branch timeout, fold stall, vessel end, sim error, **and** `max_steps`). The buffer stores only `episode.terminals` (`replaybuffer.py:78-84`; `truncations` are dropped at `to_replay()`), and the critic target masks with it: `sac.py:587` `expected_q = rewards + (1 - dones) * gamma * next_target_q`. Consequence: for a pure time-limit truncation the target is `r` with no bootstrap, so `Q` at late-episode states learns "the world ends here" — states visited late get systematically pessimistic values even when the state itself is healthy. The comment's rationale (98% of ends are failures; critic diverged without an anchor) is coherent for the *failure* truncations, and the `-5 FailureTruncationPenalty` fires first, but `max_steps` and `sim_error` are not MDP failures. Fix: keep `terminated=True` for the failure-classified truncations only; for `max_steps` / `sim_error` store `terminated=False` and *also* store truncation so the buffer can mask correctly (or bootstrap through timeouts, the standard `TimeLimit` handling). Because `Episode.truncations` never reaches the buffer, this distinction currently cannot be recovered downstream.

A2. BUG (dead knob) — `reward_scaling` is stored and never applied. `sac.py:155,201` accept and store `reward_scaling`; it is never referenced in `update()` / `_get_expected_q` (the target at `sac.py:587` uses raw `rewards`). Same in `iql.py`. It's plumbed all the way from `DualDeviceNav_train.py:70` (`REWARD_SCALING = 1`) through `util\agent.py:299`. Today the value is 1 so behavior is right, but anyone tuning this flag gets a silent no-op, and checkpoint configs record a knob that does nothing. Fix: apply `rewards * self.reward_scaling` in `_get_expected_q` or delete the parameter.

A3. Checked-correct: `torch.narrow(all_states, 1, 0, seq_length) / narrow(next_target_q, 1, 1, seq_length)` (`sac.py:282,586`) align current/next states correctly for the (B, 2, obs) step layout;

rewards/dones shapes `(B,1,1)` broadcast correctly; `expected_q.detach()` used in both critic losses; gamma applied once, no double-discounting; no n-step logic anywhere (pure 1-step); no reward normalization anywhere.

B. AWAC actor loss (checklist 2)

B1. CONCEPTUAL/BUG-risk — `log_prob` floor clamp kills the BC gradient for exactly the demos AWAC needs most. `network\gaussianpolicy.py:76`: `return log_prob.sum(-1, keepdim=True).clamp(min=-20.0)`.

`clamp` has zero gradient in the clamped region, so any buffer action whose log-prob under the current policy is below -20 contributes **zero policy gradient** (the AWAC term `-(weight*logp)` in `sac.py:446` becomes a constant). With 4 action dims and `log_std` railed at -2 ($\text{std} \approx 0.135$), a demo $\sim 3\sigma$ from the mean per dim already sums below -20 . Failure mode: once the policy drifts away from the demo manifold, the high-advantage demos silently stop pulling it back — a self-reinforcing freeze that would look like "AWAC stopped learning" while `awac_weight_*` diagnostics still look fine (the weights are computed from Q, not logp). Fix: clamp with a straight-through/soft floor (e.g. `-softplus(-(logp+20))-20`), or clamp per-dim `z` distance instead, and log the clamp-hit fraction.

B2. BUG (minor, masked-batch only) — padding-mask bias in advantage. `sac.py:406-407` multiplies `min_q` by `padding_mask` before it is used as the V(s) baseline at `sac.py:420` (`advantage = q_buf - min_q.detach()`), while `q_buf` (417-419) is unmasked. At padded timesteps `advantage = q_buf - 0` → `weight = exp(q_buf/λ)` clamped at 20. The subsequent `weight * padding_mask` (436) zeroes the loss contribution, so no wrong gradient — but the `_awac_weight_saturation/max/mean` diagnostics (428-432) are computed **before** masking and get polluted by garbage weights. Only affects episode-mode (step-mode `padding_mask=None`). Fix: compute the diagnostics after masking, or mask `q_buf` too.

B3. Checked-correct (design choices worth knowing): - Advantage form is $Q(s, a_{\text{buf}}) - Q(s, a_{\pi})$ with a **single** policy sample as V(s) (`sac.py:401-420`) — the standard single-sample AWAC baseline (original paper averages ≥ 1 samples; 1 is accepted, just higher-variance). - Per-sample `exp(A/λ)` ($\lambda=3.0$), not softmax-over-batch — matches AWAC paper. - Weight clipped at 20 (`sac.py:421`), detached, no grad through advantage. Correct. - The added entropy bonus `+ alpha.detach() * log_pi` (`sac.py:446`) uses the tanh-corrected `log_pi` from `_get_update_action` (`sac.py:603`: `normal.log_prob(z) - log(1-tanh(z)2+ε)`), i.e. it is the entropy of the squashed policy — correct w.r.t. the Jacobian. `alpha` is detached in the AWAC branch (its gradient comes from `_update_alpha`). No double-counting: the AWAC critic target deliberately has no entropy term (`sac.py:572-579`), so entropy appears exactly once, in the actor loss. - The optional per-dim `entropy_beta` / `action_mean_penalty` regularizers (`sac.py:471-488`) are off in the current launch (not passed).

B4. INEFFICIENCY — 3 redundant policy forwards + 1 wasted log-prob per AWAC update. Per `update()` the policy body is forwarded: (i) `_get_expected_q → _get_update_action(all_states)` (`sac.py:566`), (ii) `_update_policy → _get_update_action(states)` (401), (iii) `policy.log_prob(states, actions)` (433, another full forward), and (iv) a *fourth* forward if the regularizers are on (475). `states` is a slice of `all_states`; (ii) could be sliced out of (i)'s forward, and (iii) could reuse (ii)'s `mean/log_std`. Also `next_log_pi` from (i) is computed and discarded in the AWAC branch. Roughly 2 policy forwards per update are avoidable — a few % of trainer step time.

C. log_std_min / log_std_max caps (checklist 3)

C1. CONCEPTUAL (high relevance to the v2 experiment) — hard `torch.clamp` makes both rails sticky.

`gaussianpolicy.py:44,53`: `log_std = torch.clamp(log_std, min, max)`. Gradient is exactly zero outside the band, so once the pre-clamp head output crosses a rail, **no loss term can push it back directly** — not the `alpha*log_pi` entropy term, not the per-dim entropy betas (which act on the clamped value, `sac.py:475-482`), nothing. Only incidental trunk drift can un-rail it. With the original $(-20, 2)$ band this rarely matters; with the deliberately narrow $(-2, 0)$ band of `launch_lcca_awac_v2.sh:100-101` railing is *expected* (the v1 forensic saw `log_std` $\rightarrow +2$ on all dims — i.e. hard-railed at the old max). The new `max=0` cap will stop the variance explosion, but by saturation, not regulation: pre-clamp values will pile up above 0 with zero restoring gradient, and the "log_std" telemetry will read a frozen 0.0. Fix: rescale instead of clamp — `log_std = log_std_min + 0.5*(log_std_max-log_std_min)*(tanh(x)+1)` (SAC-v2 style) so gradient exists everywhere; entropy/log-prob stay automatically consistent because downstream always uses the returned (bounded) `log_std`.

C2. Checked-correct: everything downstream (`std = log_std.exp()`, `Normal`, entropy, `log_prob`) consistently uses the *clamped* value in every path — sampling (`sac.py:596-604`), AWAC `log_prob` (`gaussianpolicy.py:66-70`), exploration/eval (`sac.py:230-251`). No path uses unclamped std, so there is no train/act inconsistency — the problem is purely the dead gradient of C1.

D. PER (checklist 4)

D1. BUG (low frequency) — stale-index priority updates can hit ring-overwritten or prefetch-raced slots. The buffer subprocess **eagerly pre-samples** a batch whenever the sample queue is empty (`pervanillashared.py:153-155`), so the batch a trainer consumes was drawn ≥ 1 batch before its own priority feedback is applied, and the `(indices, td)` it sends back (`single.py:686-697` $\rightarrow$ `pervanillashared.py:33-50` $\rightarrow$ `pervanillastep.py:309-326`) may refer to slots that a worker `push()` has since overwritten (`pervanillastep.py:154-171` ring). The new transition then inherits the old transition's TD priority. With capacity 11 M this only bites after wrap-around, and the subprocess drains priority updates before pushes (`pervanillashared.py:148-151`) which shrinks the window — but it's a real, silent corruption channel on long runs / smaller buffers. Cheap fix: stamp each slot with a monotonically increasing `write_id`, return it with the batch, and drop priority updates whose id no longer matches.

D2. CONCEPTUAL — beta never reaches 1.0. `per_beta_steps = float(TRAINING_STEPS) = 2e7` (`DualDeviceNav_train.py:183`), annealed per `sample()` call (`pervanillastep.py:259-265`), but the run does $\sim \text{TRAINING_STEPS} \times \text{update_per_explore_step} = 1e7$ updates (`launch...sh:102` sets 0.5), and samples $\approx$ updates (+1 prefetch). Final `beta` $\approx 0.4 + 0.6 \times (1e7/2e7) = \mathbf{0.7}$, so the IS correction never becomes unbiased. If the intent was "anneal over the run", `beta_steps` should be `TRAINING_STEPS  $\times$  update_per_explore_step`.

D3. NIT — IS weights normalized by batch max, not buffer max. `pervanillastep.py:291` `weights /= weights.max()`. The PER paper normalizes by the max weight over the buffer (keeps all weights ≤ 1 with a stable scale); batch-max normalization injects batch-to-batch effective-LR jitter (a batch of uniformly high-priority samples gets the same scale as a mixed batch). Common shortcut, worth knowing when reading loss curves.

D4. CONCEPTUAL (documented) — two-stream IS weights are wrong w.r.t. the true mixture. `_draw` (`pervanillastep.py:239-256`) computes `w = (N·P_stream)^(-β)` per stream; the true sampling probability

under balanced mixing is `0.4·P_gen + 0.6·P_clean(i∈clean)`. Clean transitions are heavily over-sampled (by design) but only IS-corrected against their own stream, so the "correction" does not remove the up-weighting bias. That's the intent (it's an up-weighting mechanism), but it means β -annealing does *not* converge this estimator to unbiased — don't expect PER theory to hold for the clean lane.

D5. BUG (rare crash/contamination) — `SumTree.get_leaf` boundary handling. `pervanillastep.py:51-63`: with `value <= tree[left]` and `random.uniform` able to return exactly a segment boundary (including 0.0), a draw can descend into a **zero-priority** subtree (e.g. leftmost leaves of the `clean_tree` where most leaves are 0, or unfilled leaves beyond `len(buffer)` when the buffer isn't full). Consequences: sampling a non-clean transition from the clean stream (mild), `prob→0 → weight=(n·1e-12)^-β` enormous pre-normalization (skews the whole batch after D3's batch-max normalization), or `self.buffer[leaf_idx]` `IndexError` killing the buffer subprocess if the leaf is beyond the filled region. Standard fix: after `get_leaf`, retry/clamp when `priority == 0`, and use `value < tree[left]` with an epsilon guard.

D6. Checked-correct: IS weights ARE applied to both critic losses (`sac.py:540-544, 514-518`) and (harmlessly, matches several public impls) to the policy loss (453-456); priorities are refreshed from post-update `|TD|` of q1 (`sac.py:540-549` → `single.py:686-697`), demo bonus re-applied on refresh with a monotone raw-space `max_priority` and `α` applied once (`pervanillastep.py:135-145, 314-326`); new pushes enter at running max (correct "sample at least once" behavior); `indices` order (general + backfill + clean) matches the batch stacking order so TDs land on the right leaves; sum-tree `update()` delta propagation is correct; `export_all/import_all` round-trips tree, flags, `max_priority`, `_sample_count`, and rebuilds `clean_tree` from `is_clean` × leaf priorities.

D7. NIT — `import_all` docstring says "skip the load defensively", code raises (`pervanillastep.py:207-216`). The raise is the better behavior; fix the comment.

D8. NIT/dead-code — `bulk_import_arrays` + `_recompute_all_priorities` have zero callers (searched `eve_rl` + `training_scripts`). `bulk_import_arrays` leaves the sum-tree all-zero (`pervanillastep.py:328-365`); a caller that forgets the documented `_recompute_all_priorities()` gets an empty-batch crash in `sample()`. If the offline path is resurrected, make `bulk_import_arrays` call the recompute itself.

E. Target networks (checklist 5)

E1. Checked-correct: Polyak `target ← τ·online + (1-τ)·target` with $\tau=0.005$ every update (`sacmodel.py:91-100`, called `sac.py:308`); target forwards are under `torch.no_grad()` (`sac.py:568-570`); targets are deep-copied at init and their state dicts are saved/loaded in checkpoints (`sacmodel.py:118-150`) — no "fresh critics on warm start" hole.

E2. INEFFICIENCY (tiny) — polyak math builds autograd graph. `tau * param + (1 - tau) * target_param` (`sacmodel.py:95,100`) computes on `Parameter`'s with `requires_grad=True` outside `no_grad`; the temporary graph is created and discarded every update ×(all target params). Wrap in `torch.no_grad()` and use `target.mul_(1-τ).add_(param, alpha=τ)`.

E3. INEFFICIENCY (latent) — `_get_expected_q`'s policy sample is outside `no_grad`. `sac.py:566`: `next_actions, next_log_pi = self._get_update_action(all_states)` builds a full policy autograd graph whose outputs are consumed only inside a `no_grad` block / a `.detach()` ed target. Per-update wasted graph

construction + activation memory for a (B,2,obs) forward. Wrap the call in `torch.no_grad()` (both SAC and AWAC branches are safe: `expected_q` is always `.detach()`ed at use).

F. Entropy temperature α (checklist 6)

F1. BUG-risk — `log_alpha` is unbounded, and AWAC + the `log_std` ceiling make runaway plausible.

`sacmodel.py:63` `log_alpha = zeros(1)`, no clamp anywhere; `_update_alpha` (`sac.py:389-398`) will grow it whenever policy entropy sits below `target_entropy=-4`. In AWAC the entropy the policy can actually reach is capped from above by `log_std_max=0` (C1) and pulled down by the BC term; if the equilibrium entropy under BC pressure sits below -4 (very possible with 4 dims and demos requiring precise actions), `log_alpha` integrates upward without bound until `alpha*log_pi` swamps `weight*logp` — the entropy bonus then bulldozes the BC objective (mean $\rightarrow 0$, maximal-entropy policy) and eval collapses "mysteriously". Symmetrically it can collapse toward 0. Fix: clamp `log_alpha` (e.g. to $[-10, 2]$) after the optimizer step, and alarm on `alpha` in `last_metrics` (it's already logged — add a watchdog threshold).

F2. BUG (episode-mode only) — masked `log_pi` biases the alpha loss. `_update_policy` returns `log_pi * padding_mask` (`sac.py:408,506`) and `_update_alpha` computes `(-log_pi - target_entropy)` over all positions (390-392); padded entries contribute the constant `+n_actions` coefficient, pushing `log_alpha` down. None in step mode (mask is None). Fix: exclude padded entries (masked mean).

F3. NIT — SAC branch doesn't detach alpha in the policy loss. `sac.py:449` `self.alpha * log_pi - min_q` where `self.alpha = log_alpha.exp()` from the previous `_update_alpha` (397) still carries its graph. `policy_loss.backward()` therefore deposits spurious grad into `log_alpha`; it happens to be harmless **only** because `_update_alpha` calls `zero_grad()` before its own backward (393). Fragile ordering dependency — use `self.alpha.detach()` in the SAC branch too (the AWAC branch at 446 already does).

F4. BUG (minor) — stale `self.alpha` on the first update after a warm start. `load_state_dicts_network` restores `log_alpha` (`sacmodel.py:150`) but `self.alpha` (init `ones(1)`, `sac.py:209`) is only refreshed at the end of an update (397). The first update after loading a checkpoint uses $\alpha=1.0$ regardless of the saved temperature (both in the SAC critic target 583 and both policy branches). One-step glitch; refresh `self.alpha` inside `load_state_dicts_network` or lazily from `log_alpha` at the top of `update()`.

F5. Checked-correct: alpha-loss sign/form `log_alpha*(-log_pi-H).detach()` is the standard estimator; `target_entropy = -n_actions` (`sac.py:210`) is the SAC default; alpha optimizer state is saved/loaded (`sacmodel.py:157-171`).

G. balanced_fraction / demo mixing (checklist 7)

G1. Checked-correct: proportions — `n_clean = round(0.6*B)`, `n_general = B - n_clean`, backfill from general when a stream comes up short (`pervanillastep.py:270-291`); `VanillaStep` equivalent at `vanillastep.py:55-74`. Demo/clean transitions carry the same `(flat_obs[i:i+2], action, reward, terminal)` tuples as everything else (`pervanillastep.py:154-171`), so `done/next_obs` are right; `is_demo / is_clean / episode_return` flow `Episode.to_replay()` (`replaybuffer.py:61-84`) $\rightarrow$ push $\rightarrow$ per-slot arrays, and the heatmap-cache loader (`DualDeviceNav_train.py:773-799`) reconstructs `EpisodeReplay` with

the saved metadata. The strict `final_branch == target` clean definition (with the R17 latch fix) is applied consistently at save (`runner.py:554-635`) and replay-push time.

G2. NIT — clean-lane duplicates. Clean transitions live in both trees, so a batch can contain the same transition twice (once per stream). Intended up-weighting; just don't interpret batch-unique counts from it.

H. Multi-worker / staleness / seeds (checklist 9)

H1. CONCEPTUAL — workers run up to a full explore cycle (~100 episodes) on frozen weights. Weight sync happens once per `explore_and_update` cycle, *after* both trainer and workers finish (`synchron.py:611-612`); a cycle is `CONSECUTIVE_EXPLORE_EPISODES = 100` episodes (`DualDeviceNav_train.py:65`) across 16 workers. At SOFA-episode lengths that's tens of thousands of env steps and (at ratio 0.5) thousands of gradient updates of staleness. Off-policy SAC/AWAC tolerates this, but it fattens the behavior/target policy gap exactly when the policy is changing fastest (right after pretrain). Consider syncing mid-cycle (e.g. per N episodes via the existing model queue) or shrinking the cycle.

H2. Checked-correct (races/seeds): the actual buffer is single-owner in its own subprocess; workers/trainer only touch `mp` queues (`vanillashared.py`, `pervanillashared.py`) — no shared-structure races. Push/sample/priority/task interleaving is serialized by the loop; priority updates drain before pushes (`pervanillashared.py:148-151`). Worker RNG: spawn-time reseed of numpy/random/torch per worker from `(base, worker_id, pid)` (`singelagentprocess.py:135-145`) plus the per-(worker, episode) `default_rng` for heatup with the v13 persistent-generator fix (`single.py:334-388, 462-469`) — the earlier identical-stream and frozen-action bugs are genuinely fixed. Trainer→parent weight sync via shared-memory `destination` copy (`singelagentprocess.py:264-272` + `sacmodel.py:128-137`) is blocked-on via the model-queue round-trip, so no torn reads.

H3. INEFFICIENCY — weight distribution does a redundant full serialization + fixed sleeps. Each sync also pickles the full CPU state dict back through `model_queue` only to be discarded by `_update_algo_state_dicts` (`synchron.py:636-638`), and `_worker_load_state_dicts_network` sleeps 0.1 s per worker (`synchron.py:101-104`) = 1.6 s/cycle for 16 workers. Minor at current cycle lengths.

H4. BUG-ish — exploration noise is a *scalar* shared across action dims, and unclipped. `sac.py:235` (and `SACPlayOnly:82,96`, `iql.py:70`): `action += np.random.normal(0, self.exploration_action_noise)` — `np.random.normal` without `size` returns one float, so the **same** offset is added to all 4 dims (gw_trans, gw_rot, cath_trans, cath_rot) each step: exploration noise is perfectly correlated across devices/DOFs, which is almost certainly not intended and shrinks the explored action manifold. Also the result can leave $[-1, 1]$; `_play_episode`'s normalization (`single.py:112-117`) then produces env actions beyond `action_space.high/low`, and the out-of-range action is what gets stored in the buffer. Fix: `np.random.normal(0, σ, size=action.shape)` and `np.clip(action, -1, 1)`.

I. Checkpoint save/load (checklist 10)

I1. BUG — `env_train` config is saved from `env_eval`. `agent\agent.py:326-335`: both `env_eval_config` and `env_train_config` call `self.env_eval.get_config_dict()` (and both `hasattr` checks probe `self.env_train`). Any `from_checkpoint` that reconstructs `env_train` from the file silently gets the eval env

(different target sampling/curriculum). Currently masked because your launchers pass envs explicitly, but it corrupts the provenance record in every `.ever1` written.

12. BUG (latent, non-PER runs) — replay-buffer save/load deadlocks uniform buffers.

`VanillaStepShared.loop` handles `save_buffer` / `load_buffer` by calling `internal_replay_buffer.export_all()` / `import_all()` (`vanillashared.py:227-240`), but **neither** `VanillaStep` **nor** `VanillaEpisode` **defines those methods** (verified: no `export_all/import_all` in either file). `runner.eval` calls `save_buffer_to_file` every eval (`runner.py:347-354`); on a non-PER run the subprocess dies on `AttributeError` and the caller blocks forever in `self._result_queue.get()` (`vanillashared.py:125-129` — no timeout), freezing training at the first eval. The runner's try/except can't help because the exception is in another process. PER runs are unaffected (`pervanillashared.py:168-197` implements both). Fix: implement `export_all/import_all` on the uniform buffers or guard the task with a capability check + error result.

13. Gap — resume never restores the replay buffer or runner state in the train script.

`Runner.load_replay_buffer` / `restore_runner_state` exist (`runner.py:108-164`) but `DualDeviceNav_train.py` only ever calls `agent.load_checkpoint` on the `--eval_only_checkpoint` path (line 517-552). A "warm start" of training from a `.ever1` therefore resumes networks/optimizers/counters but trains on an empty (or freshly re-seeded) buffer with PER priorities reset — a silent distribution shift vs. the original run. If intentional (fresh-buffer restarts), fine, but document it; the counters restored from the checkpoint (`agent.py:373-380`) also interact with `explore_and_update`'s budget formula (`runner.py:395-399`) — a resumed run with restored `update` count but zeroed `_pretrain_update_baseline` computes `max(0, explore_ratio - update)` = 0 updates until exploration catches up. That's a real resume trap.

14. Checked-correct: `.ever1` includes networks (incl. **both targets** and `log_alpha`), all four optimizers, schedulers, counters (`agent.py:321-359`; `sacmodel.py:118-171`); `Synchron.load_checkpoint` propagates nets+optim+schedulers to the trainer and nets to all workers (`synchron.py:246-248, 749-753`); optimizer state loaded after `to(device)` is device-cast correctly by PyTorch. Worker `SACModelPlayOnly` loads only the policy — correct for play-only.

15. NIT — `SAC.to_play_only` **drops** `stochastic_eval` (`sac.py:762-768`): workers (which run *all* evals in `Synchron`) always evaluate deterministically even if `--stochastic_eval` was set. Currently default-False so no impact.

J. Dead code, wasted compute (checklist 11)

J1. BUG (latent) / INEFFICIENCY — optimizers over the wrong module set in `training`

`_scripts\util\agent.py`. `q1_optim = Adam(q1)` (full net) but `q2_optim = Adam(q2_base)` and `policy_optim = Adam(policy_base)` (bodies only) — lines 196-233 (`Synchron`) and 49-78 (`Single`). The **same** `q1_embedder` instance is the head of `q1`, `q2`, and the policy (195, 207, 219). With `--embedder_layers 0` (current run) the head is a parameterless `Dummy`, so no impact. But the moment an embedder is enabled: (a) it's stepped only by `q1`'s optimizer while receiving gradients from all three losses; (b) `zero_grad` on `q2/policy` never clears embedder grads, so `q1`'s step uses embedder grads contaminated by the previous `q2/policy` backwards is avoided only by ordering (`_update_q1` zeroes first), while `clip_grad_norm(q2.parameters()) / (policy.parameters())` (`sac.py:499-502, 525-528`) *do* include the stale embedder grads in the norm they clip against — distorting the clip scale for the body grads; (c)

`compute_grad_norm` diagnostics are similarly contaminated. Fix: give each net its own head (or explicitly share intentionally with one owner + `requires_grad` discipline), and build every optimizer over the full network.

J2. INEFFICIENCY (real, current run) — MLP has two stacked Linear layers with no activation between.

`mlp.py:83-87` (# TODO: Add F.relu after input layer): `_input_layer` output feeds `_layers[0]` directly. With `hidden = [256, 256]` every net is effectively `Linear(obs→256)◦Linear(256→256)` ($\approx$ one rank- ≤ 256 linear map) $\rightarrow$ ReLU $\rightarrow$ heads, i.e. **one** nonlinear hidden layer where two were configured. All three nets in the AWAC run are shallower than intended, and 65k parameters per net are near-redundant. One-line fix; note it changes the function class, so results aren't comparable across the fix.

J3. Dead/broken — `component\lstm.py`. `forward` computes `state = [layer(state) ...]` with `state` **undefined** (`NameError`) when `_output_layers` exist, then returns `output` ignoring them anyway (`lstm.py:85-90`); `device` property references nonexistent `self.lstm` (82-83). Works only as a head (no output layers). Also `SACPlayOnly/SAC.get_eval_action` call `forward_play` **twice** in the deterministic branch (`sac.py:244-251, 105-112`) — with an LSTM head this advances `_hidden_state` twice per env step (corrupted recurrent eval); with the MLP/Dummy head it's just a wasted duplicate forward per eval step. Delete the second call (`mean` is already in scope).

J4. INEFFICIENCY — **per-update sync storm in diagnostics.** `compute_grad_norm` does a `.item()` per parameter tensor, called $3\times$ per update (`sac.py:13-21, 493, 523, 554`), plus ~ 15 further `.item().cpu()` calls in the metrics block (320-381), all unconditional (not gated on diagnostics being enabled). Each is a CUDA sync. On a small model this can be a double-digit % of trainer-step wall time. Batch them (`torch._foreach_norm`, or compute norms from `clip_grad_norm`'s return), and gate on `self.diagnostics`.

J5. Dead code — `Batch.to()` (`replaybuffer.py:135-178`) is unused (SAC does its own `.to`), and would be harmful if used: `.share_memory_()` on CUDA tensors is a no-op but on CPU it forces an extra shared-mem allocation per field per batch. Also `SingleEvalOnly._next_batch/_replay_too_small` on the eval-only class (`single.py:33-34`), `get_action_exploration/get_action_evaluation` returning tuples against an `np.ndarray` signature (`sac.py:85-97, 117-133`), and the whole `*_old*.py` family (`sac_old, sac_old2, single_old, single_old2, synchron_old, runner_old, runner_old2, singelagentprocess_old, vanillashared_old, __init__old, diagnostics_logger_old2`) still ship in the package.

J6. NIT — `mp.Value('q')` comment says "unsigned long long" (`vanillashared.py:52`); `'q'` is signed (unsigned is `'Q'`). Harmless.

J7. NIT — `EpisodeCounter.lock` / `StepCounter.lock` are class attributes (`agent.py:21,36` — no annotation in the dataclass), shared by all instances in a process. Works today (one agent per process), but any second agent instance shares the lock.

K. Eval mode (checklist 12)

K1. Checked-correct: deterministic eval is `tanh(mean)` (`sac.py:249-251`) — correct mode of the squashed Gaussian for reporting; `action_scaling=1`; eval seeds are deep-copied and split deterministically across workers (`synchron.py:71-85`). No dropout/BatchNorm anywhere in MLP/LSTM/Q-nets, so train/eval mode leakage is impossible in the current architecture (target nets' `.eval()` is decorative). Worker eval uses freshly synced post-cycle weights.

K2. See J3 (double `forward_play` per eval step) and **I5** (`stochastic_eval` dropped in `to_play_only`).

Priority summary

#	Finding	Class	Bite risk now
C1	Hard clamp on log_std with narrow (-2, 0) band → sticky rails, dead entropy gradient	CONCEPTUAL	High — it's the exact mechanism the v2 experiment leans on
F1	Unbounded <code>log_alpha</code> + AWAC entropy bonus + capped entropy ceiling → alpha runaway	BUG-risk	High
B1	<code>log_prob.clamp(min=-20)</code> zeroes BC gradient on far demos	CONCEPTUAL/BUG-risk	High during recovery-from-collapse phases
J2	MLP missing ReLU after input layer — nets are 1 hidden layer, not 2	INEFFICIENCY (capacity)	High (affects every run)
H4	Scalar, unclipped exploration noise shared across all 4 action dims	BUG	Medium (online phase)
A1	max_steps truncations grounded as terminal	CONCEPTUAL	Medium (value bias)
D2	PER beta ends ≈0.7, never 1.0	CONCEPTUAL	Medium
I2	Buffer save deadlocks non-PER runs at first eval	BUG (latent)	High if you ever run <code>-- replay_mode step</code> without <code>-- per</code> or episode mode
I3	Training resume never restores buffer/runner state; update-budget trap	GAP	Medium on any warm-start relaunch
J1	q2/policy optimizers exclude (shared) embedder	BUG (latent)	High if <code>embedder_layers > 0</code> ever
D1, D5	PER stale-index race; zero-priority leaf edge	BUG (rare)	Low-medium
E3, B4, J4, J3	wasted forwards/graphs/syncs per update	INEFFICIENCY	steady trainer-throughput tax
F2-F4, B2, I1, I5, D3, D7, J5-J7, G2	remaining items	NIT/minor	low

Everything in the checklist was verified against code rather than assumed; the "Checked-correct" bullets (A3, B3, C2, D6, E1, F5, G1, H2, I4, K1) enumerate what I explicitly confirmed clean.

Training harness/config review

Agent: Training harness/config review · Source file: 3.md

Review: LCCA AWAC v2 training orchestration (active run)

Executive summary

The v2 launcher's flags all parse, all reach the code, and the seed buffer on disk matches the handoff's claims exactly. The uncommitted diff is purely the handoff-`$4` mount-prefix rewrite. The two headline experiments (EXP-1 `--log_std_max 0.0`, EXP-2 `--balanced_fraction 0.6`) are correctly plumbed end-to-end. However, I found **two real bugs in the eval/exploration machinery** (eval-seed loss on worker restart; 16 workers sharing one identical explore-target RNG sequence), one **growing per-eval multi-GB buffer dump**, and several doc-vs-code drifts — including the claim that LCCA v1/v2 mirror the RCCA-v3 "86% stack byte-for-byte," which is false on three hyperparameters.

BUGS

B1. Worker crash during eval silently corrupts the Quality denominator

`eve_rl/eve_rl/agent/synchron.py:87-88` — `Synchron.evaluate()` nulls `self._eval_seeds` / `self._eval_options` immediately after dispatch, *before* `_get_worker_results()`. If a worker dies mid-eval, `_restart_worker_agent(..., task="evaluation")` (`synchron.py:646-675`), and the same pattern at `synchron.py:203-227` reads `seeds = self._eval_seeds[i] if ... is not None else None` → **None**. The restarted worker's eval loop (`eve_rl/eve_rl/agent/single.py:58-89`) then plays exactly **one unseeded episode** (breaks on `not seeds`) and returns. Effects: (a) the dead worker's 6–7 seeded episodes are lost (its completed results are also lost — results return only at task end), (b) one random-target, unseeded episode enters the Quality average. Additionally the 70-min hard timeout (`synchron.py:110-138`) returns *partial* results. `Runner.eval()` (`eve_rl/eve_rl/runner/runner.py:279-315`) averages over whatever comes back — the "98-seed" Quality can silently be a ~91-episode number with a random episode mixed in. Worker restarts demonstrably happen ("Restarting Agent ... because of Timeout"). **Fix:** keep `_eval_seeds` alive until results are collected, and re-dispatch the dead worker's *remaining* seed list; log the actual episode count next to `Quality:`.

B2. All 16 workers draw the IDENTICAL explore-target sequence

`eve/eve/intervention/target/centerlinrandom.py:34` — `self._rng = random.Random()` is created once in the **main process** when the env is built. Each worker gets the env via `deepcopy(self.env_train)` / spawn-pickle (`synchron.py:693-714`), which **copies that RNG state verbatim**. Explore episodes reset with `seed=None` (`single.py:639-643`), and `monoplanestatic.py:143-167` only reseeds the target RNG when a seed is given — so every worker's *j*-th explore episode samples the **same LCCA target point**. The Plan-v12 R16 spawn-time reseed (`eve_rl/eve_rl/agent/singelagentprocess.py:125-145`) fixes numpy/`random` - module/torch **globals** only; the pickled private `random.Random` instance is untouched. Worse, a restarted worker replays the target sequence from the beginning. Net effect: explore-target diversity is ~1/16 of nominal per wall-clock window and PER receives 16-way-correlated data. (Eval is unaffected — eval seeds reseed the RNG per episode. The Plan-v12 heatmap harvest was also mostly unaffected because secondary-

grader target probes are built lazily *inside* each worker.) **Fix:** reseed `intervention.target._rng` per worker (e.g., in the R16 spawn block or on first unseeded reset), same recipe as R16.

B3 (latent). `--stochastic_eval` can never reach the eval workers `eve_r1/eve_r1/algo/sac.py:762-768` — `SAC.to_play_only()` constructs `SACPlayOnly` without forwarding `stochastic_eval`; workers (`synchron.py:699-714`) therefore always eval with `tanh(mean)`. Harmless for this run (deterministic eval is intended and the flag is off), but the flag is silently ignored if anyone flips it.

B4 (latent). `log_std_min/max` threaded into only 1 of 3 policy construction sites `training_scripts/util/agent.py:219-223` (`BenchAgentSynchron`) passes them; `agent.py:72-74` (`BenchAgentSingle`) and `agent.py:421-423` (`create_bench_agent`) still build `GaussianPolicy` with defaults ($-20/+2$). Active run uses only `BenchAgentSynchron` → OK, but this **contradicts RL_IMPROV_10_CHANGES.md §13a** ("threaded it into all three `GaussianPolicy(...)` calls").

INEFFICIENCIES

I1. Full replay-buffer export on every eval — grows to multi-GB, synchronous `runner.py:60` (`_replay_save_interval = 1`) + `runner.py:347-354` + `pervanillashared.py:168-185` + `pervanillastep.py:173-201` (`export_all` does `np.stack` of the whole buffer then uncompressed `np.savez`). With `--replay_buffer_size 11000000`, each 250k-step eval writes ~0.7 GB at seed-only fill, rising toward ~7 GB as the buffer fills, doubling that in the buffer subprocess's RAM during `np.stack`, and blocking the main loop (called under `_request_lock`). The interval=1 choice was calibrated for the RCCA 1M buffer. **Fix:** raise `_replay_save_interval` (or gate on buffer size) for 11M-buffer runs.

I2. One snapshot PNG per episode, no rate limit in single-target mode `training_scripts/util/env5.py:1311-1332` — the internal end-of-episode snapshot fires for every terminated episode; `SNAPSHOT_EVERY` (`train.py:293`) is only consumed by the MultiTargetEnv5 driver (`env5.py:1901-1904`). A 20M-step run ⇒ O(100k) PNGs unless `prune_training_snapshots.py` is run manually. Same as RCCA v3, so presumably accepted — but it accumulates for the whole relaunch.

I3. GPU idle during eval; policy weight lag during explore Eval is fully synchronous in `training_run` (`runner.py:752-763`), and worker policy weights refresh only at `explore_and_update` cycle boundaries (~100 episodes ≈ 20-60k steps, `synchron.py:611-612`). Known framework property, noted for completeness.

CONCEPTUAL

C1. Eval denominator/statistics — 98 seeds (verified: 98 unique; `DualDeviceNav_train.py:60-62`), one deterministic episode each, Quality = mean of `info["success"]` (TargetReached, 5 mm threshold, `runner.py:279-301`, `dualdevicenav.py:144-150`). Binomial SE: ±1.7pp at p=0.03, ±2.6pp at p=0.07, ±5.1pp at p=0.5. The v2 decision question ("3% vs 7%") is ~1.5σ on a single eval — only trends over several evals are meaningful (the handoff already says this, good). `best_checkpoint.ever1` selection (`runner.py:306-309`, `333-337`) is argmax over this noisy metric — fine for monitoring, but don't report the best-checkpoint number as the result.

C2. Train/eval termination mismatch — `env5.py:346-351`: train truncation = MaxSteps+VesselEnd+SimError; **eval = MaxSteps only**. The custom off-path-50 and fold-stall-20 detectors run in both (`env5.py:837-871`, `1015-1064`), so most failures still truncate in eval, but: eval episodes can't

vessel-end truncate (in practice they die via fold-stall since devices stop at tree end), and a SOFA `sim_error` in eval produces a zombie episode that steps a broken sim to 600 and counts as failure. Mildly asymmetric; long-standing/deliberate, not new in v2.

C3. PER beta anneal unit mismatch — `per_beta_steps = TRAINING_STEPS = 2e7` (`train.py:183`) but beta advances per `sample()` call = per update (`pervanillastep.py:259-265`), and `--update_per_explore_step 0.5` means only ~10M updates over 20M explore steps → beta ends at 0.7 even for a full run; at the intended ~2M-step stop, beta ≈ 0.43 (IS correction stays weak throughout). Same in v1, so it doesn't confound the v1↔v2 comparison.

C4. TRAINING_STEPS=20M vs "run to ~2M" — `train.py:64`; the run never self-terminates near the handoff's ~2M goal; it must be `docker stop` ped. LR schedule (6M updates to 0.15×, `train.py:77-78`) and beta anneal are both scaled to the 20M assumption.

C5. Exploration noise is a single scalar added to all 4 action dims — `sac.py:224-236` (`action += np.random.normal(0, 0.25)`, no `size`): correlated across dims and can exceed [-1,1] before denormalization (`single.py:111-117`). Pre-existing library quirk affecting all runs equally; the stored buffer action is the noised, unclipped value (AWAC `log_prob` clamps at $\pm(1-\epsilon)$, `gaussianpolicy.py:68`).

NITs

- `runner.py:713` — probes dead this run: `heatup_episodes` is `[]` (cache-loaded, heatup skipped), so probe states are never captured → probe CSVs/plots empty. Diagnostics parity with RCCA v3 lost.
- `train.py:852-858` — the warm-start source label prints "heatup seed this run" for the cache-load path (misleading; it's a loaded cache).
- `train.py:661` rebinds local `batch_size` inside the heuristic-seeding block (harmless — agent already constructed; block skipped in v2 anyway).
- `saved/lcca_awac_seed_v1.npz` is only rebuildable via `scratchpad/build_seed.py`, which is **not in the repo** (handoff admits this) — the curation step is unreproducible from the repo.
- Handoff §1 says Quality measures reaching "the deep LCCA target"; targets are actually sampled uniformly over the *whole* LCCA centerline pool (`centerlinerandom.py:47-51`), not only deep points.

Doc-vs-code contradictions

1. `launch_lcca_awac_v1.sh` header is the RCCA v3 header verbatim ("Plan v10 — RCCA AWAC training v3", "Pretrain 1000", "5 good restore states") — none of that describes v1. Its footer claims it "Mirrors the v3 RCCA pipeline (the 86%-on-pid19116 stack) **byte-for-byte on the frozen knobs** ... pretrain 1000", but v1/v2 actually differ from RCCA v3 in: `--pretrain_updates 10000` (v3: 1000), `--update_per_explore_step 0.5` (v3: absent → step-mode default **1.0**), `--replay_buffer_size 11000000` (v3: absent → **1M**). The buffer size is footnoted as a difference; the update ratio and pretrain count are not. The handoff repeats "This is the same stack that hit ~86%" — it is not the same update:data ratio. This matters when attributing v1's collapse or benchmarking v2 against the RCCA result.
2. **RL_IMPROV_10 §13a** vs `agent.py` — see B4.
3. Handoff §5 "eval every 250k (`EXPLORE_STEPS_BTW_EVAL=2.5e5`)" ✓ matches code; but "(many evals, ~2M steps)" is a manual-stop convention, not a configured limit (C4).

Verified correct (explicit checks)

- **Flag plumbing (all launcher flags land):** `--log_std_max 0.0` → `train.py:485` → `BenchAgentSynchron` (`agent.py:219-223`) → `GaussianPolicy.forward/forward_play` `clamp` (`gaussianpolicy.py:44,53`); applies to trainer and to worker play-only copies. `--log_std_min -2` same path. `--balanced_fraction 0.6` → `PERVanillaStepShared` → clean two-stream sampler (`pervanillastep.py:268-291`; ~154 of each 256-batch from the clean tree). `--per` → PER buffer with $\alpha=0.6$, $\beta_0=0.4$ (`agent.py:319-330`). `--grad_clip 1.0` → clip on q1/q2/policy after diagnostics norm read (`sac.py:499-503`, `525-529`, `556-560`). `--update_per_explore_step 0.5` overrides the step-mode default 1.0 (`train.py:193-196`). `--replay_buffer_size 11000000` honored (`train.py:161-170`). `--insertion_z 345` → both `DualDeviceNav(insertion_z=...)` train and eval (`train.py:360`, `433`); mutual-exclusion gates pass. `--target_branches LCCA` → `default_target_branch` injected into *both* train and eval env kwargs (`train.py:354-358`, `434-436`; `env5.py:462-466`). `awac_lambda` default 3.0 (`train.py:1344-1352`, used at `sac.py:421`) matches the handoff. Anti-rail knobs (`--entropy_beta_per_dim`, `--action_mean_penalty`) default OFF and are correctly gated (`train.py:487-493`, `sac.py:471-488`) — matching the "auto-tuned entropy is the principled anti-rail" doc. AWAC entropy auto-tune is active (`sac.py:310-316`, `446`).
- **Seed buffer:** I loaded `saved/lcca_awac_seed_v1.npz`: 8,738 episodes / 1,065,454 transitions, `meta_reached == meta_is_clean == 1012`, all `target_branch_short == LCCA`, final-branch mix LVA 3163 / trunk 2677 / LCCA 1308 / other 1264 — matches handoff §3 and the v1 footer *exactly* (including the 87 overshooters excluded from the clean lane: `meta_overshoot=87`). Exactly one terminal per episode (MDP-grounding consistent). Obs dim 78 matches the current env (any drift would crash pretrain).
- **No stale-reward seed:** `env5.py` (all reward terms) last changed in `662d5d4` (2026-06-24), the same commit as the Plan-v12 harvest tooling that produced the seed; `c7a33e3` touched only `sac/gaussianpolicy/trainer` knobs. Reward range in the npz (−5.14...+5.43) is consistent with the current config (+3 target, ±1 commits, −5/−1 truncation penalties, path-segment step costs).
- **PER pre-fill compatibility:** cache episodes enter at `max_priority` (td mode, `pervanillastep.py:135-153`) → uniform start; `is_demo=False` for the cache path (`train.py:785-795`) is harmless because `demo_priority_bonus=0`; the clean lane keys on `reached_target_daughter` = npz `meta_reached` — the intended lane.
- **Warm-start:** no checkpoint restore (matches handoff "no `--checkpoint_dir`"); the Plan-v12 pretrain gate (`train.py:842-847`) correctly accepts a heatup-cache-only seed → the 10,000 pretrain updates DO run, with the pretrain baseline subtracted from the online update budget (`runner.py:733-747`) so the first explore cycle doesn't hang.
- **Per-daughter config:** v1→v2 diff is *exactly* EXP-1+EXP-2 (flags otherwise byte-identical, mount sets identical). No RCCA copy-paste leakage in v2's *flags* (no `--checkpoint_dir`, no `--rl_start_mode`, no restore snapshots); the copy-paste is confined to v1's comment header (doc item 1).
- **Uncommitted diff:** only the mount prefix rewrite `D:\neve\claudeworktrees\rl_improv_8` → `D:\Arjun\workspace\neve` — precisely what handoff §4 prescribes; all mounted host files exist in this repo.
- **Eval mechanics:** deterministic `tanh(mean)` (`sac.py:99-115` via `single.py:65-70`), 98 unique seeds split $2 \times 7 + 14 \times 6$ across 16 workers (`synchron.py:250-257`), per-seed deterministic LCCA target (`centerlinerandom.py:40-58`, `monoplanestatic.py:163-167`), fixed worker↔seed assignment across evals, eval episodes not pushed to the buffer. Episode cap 600 steps in both modes (`env5.py:189`).

- **Worker RNG (actions/policy):** distinct per worker via the R16 spawn reseed (`singelagentprocess.py:125-145`) — except the env-target instance RNG (B2).

The single most valuable pre-flight fixes for this relaunch, in order: **B2** (explore-target RNG — costs real sample diversity every day the run is up), **I1** (`_replay_save_interval` with the 11M buffer — will progressively stall the run and hammer disk), and **B1** (log the eval episode count so a partial eval can't masquerade as a 98-seed Quality).

Env/reward/observation review

Agent: Env/reward/observation review · Source file: 4.md

I now have a complete picture. Here is the review.

Correctness + Conceptual Review — eve environment RL code (BenchEnv5 / LCCA-siphon focus)

Scope verified: active stack is `env_version=5` → `training _scripts/util/env5.py` (`BenchEnv5` / `MultiTargetEnv5`) assembling `eve.reward.TargetReached` + `eve.reward.ArcLengthProgress` + inline path-segment step reward + state-machine commit events; observation `LocalGuidance` (now **30-dim**, not 28); classification/geometry in `eve/eve/util/pathcontext.py`; projection in `eve/eve/util/polyline.py`; device/action in `eve/eve/intervention/monoplanestatic.py` + `eve_bench/eve_bench/dualdevicenav.py`; action scaling in `eve_rl/eve_rl/agent/single.py`. Confirmed by `launch_lcca_awac_v2.sh` (`--env_version 5`, LCCA target, AWAC).

1. Reward correctness

BUG-R1 — Target-daughter progress doubling breaks potential-shaping and is farmable by jitter

`eve/eve/reward/arclengthprogress.py:137-148`

On-path progress is `pf * (prev_d_rem - d_rem_curr) = pf * Δs`, i.e. a telescoping potential $\Phi = pf \cdot s$ — correct, oscillation-neutral. But Change 4b multiplies `pf *= 2.0` **only when** `delta_s > 0` **and inside the target daughter**:

```
if (delta_s > 0 and ... int(pc._current_branch_idx) == int(pc._target_daughter_branch_idx)):
    pf *= 2.0
    r_progress = pf * delta_s
```

Forward Δs earns `+2·pf·Δs`; the matching backward Δs is charged only `-1·pf·Δs` (backward has `delta_s < 0`, no doubling). **Net per oscillation cycle = $+pf \cdot \Delta s > 0$** . A wire that wiggles back-and-forth inside RCCA/LCCA accumulates unbounded positive reward without net progress (bounded only by `n_max_steps=600`). Because fold-stall requires `delta_s < 0.5` (`env5.py:858-862`), a tip that actually moves ≥ 0.5 mm each way never trips the fold detector, so the farm is unobstructed. - Failure tie-in: reinforces *shallow oscillation inside the daughter* instead of deep threading — a direct contributor to **shallow-stall-inside-daughter**. - Fix: make the depth bonus itself potential-based (e.g. add a one-time Φ -style term `pf·(s - s_enter)` credited on net new maximum depth, or drop the asymmetric doubling and raise the terminal bonus instead).

BUG-R2 — max_steps timeout carries no penalty, so stalling with banked reward is locally optimal

`env5.py:1073-1088` (failure-penalty gate) + `env5.py:1422-1423` (MDP grounding)

```
if truncated and not terminated and (
    self._vessel_end_trunc.truncated
    or self._fold_stall_count >= FOLD_STALL_STEPS
    or self._off_branch_steps >= OFF_BRANCH_GRACE_STEPS):
    reward += (OVERSHOOT_PENALTY if overshoot else FAILURE_TRUNCATION_PENALTY)
```

A pure `max_steps` truncation satisfies **none** of the three conditions, so it gets `reward += 0`, then `terminated=True` is stamped (done=True, no bootstrap). Meanwhile the episode has already banked `+1` at each on-path junction (`_maybe_emit_junction_commit`, up to +3) plus `~ pf.arclength` of shaping (`~+4`, doubled in-daughter). So a wire that reaches the daughter and simply **runs out the clock without inserting** ends net-positive (`~+7`) while never earning the +3 success — and pays nothing. The agent has a positive-value, zero-risk policy that stops short of the target. - This is, from the code alone, the single strongest structural reason success caps well below 100%: the dense front-loaded rewards (commits + progress) make the *approach* almost as valuable as the *reach*, and the terminal timeout is unpunished. - Fix: apply `FAILURE_TRUNCATION_PENALTY` (or a graded penalty by remaining `d_rem_norm`) to `max_steps` truncation too, and/or move reward mass from the junction commits to the terminal bonus.

BUG-R3 — State-machine update is one step behind every consumer that reads it (reward + off-path timeout)

`env5.py:816-817` (`super().step` runs `reward.step`) → `env5.py:872-878` (`update_branch_state`) → cache never re-invalidated

The comment at `env5.py:872-876` claims `update_branch_state()` "Must run before `is_on_correct_path()` ... so those queries see the fresh state." That is **false**. Order per step: 1. `env5.step` calls `invalidate()` (clears `_is_on_correct_path`). 2. `super().step()` → `reward.step()` → `ArclengthProgress` calls `pc.is_on_correct_path()` (`arclengthprogress.py:107`), which computes from the *previous* step's `_on_planned_path` and **caches** it. 3. `update_branch_state()` mutates `_on_planned_path` but **does not** clear `_is_on_correct_path`. 4. `_compute_path_segment_step_reward` (`env5.py:887`) and the off-path detector (`env5.py:991`) call `is_on_correct_path()` again → get the **stale cached** value.

So the shaping switch (on-path Δ s vs off-path $\Delta_{\text{off-arc}}$), the `-0.007` off-path step tax, `_off_branch_steps`, and the `-5` timeout all act on last step's classification while using this step's projection. At fork decision points (bif2, trunk-top) this 1-step lag misattributes the on/off transition. Fix: call `update_branch_state()` before reward/obs (restructure so the state machine advances at the top of `step`, driven off the just-advanced projection), or clear `_is_on_correct_path` at the end of `update_branch_state`.

CONCEPTUAL-R4 — Overshoot into the correct daughter to its distal end is penalized -5, not -1

`env5.py:1051-1062` sets `_overshoot_truncation` **only** inside the off-branch-timeout path. But a wire that threads the correct daughter and overshoots to the open vessel end trips `VesselEnd`

(`truncation/vesselend.py`) first → falls into the `self._vessel_end_trunc.truncated` arm of `env5.py:1074` with `_overshoot_truncation=False` → `-5`. So over-threading the *right* vessel is punished as harshly as diverging into a *wrong* branch. Combined with R2 (stalling = free), this actively teaches conservatism/under-reach. Relevant to **trunk under-reach** and shallow-stall.

CONCEPTUAL-R5 — Off-path is signalled by four overlapping terms

For an off-path wire: (a) `ArcLengthProgress` symmetric Δ off-arc shaping (`arclengthprogress.py:152-155`), (b) `-0.007`/step segment tax (`env5.py:1578-1579`), (c) one-shot `-0.05` deep-wrong-commit (`pathcontext.py:1225-1235`), (d) `-5` timeout. (a) is potential-based (round-trip nets ~ 0), but (b) is a **non-telescoping per-step tax** of `-0.007 * steps_off` that is paid regardless of recovery. A recovery excursion therefore has strictly negative expected value even when it succeeds, discouraging the exploratory retract-and-retry the heuristic relies on. Not a bug, but a dense-penalty design that plausibly suppresses recovery learning at forks (LVA confusion, siphon re-approach).

Reward items checked and correct

- On-path progress is a proper telescoping potential ($\Phi = pf \cdot s$); backward on-path motion is symmetric; no farm **except** BUG-R1.
- `_prev_d_rem` is updated unconditionally at `arclengthprogress.py:181`, so returning on-path yields the correct delta.
- No spurious reward at reset (reset returns obs/info only; `_prev_d_rem` / `_prev_off_arc` seeded from initial projection, re-seeded after SOFA restore at `env5.py:653`).
- `+1` junction commits are latched in `_committed_forks` (`pathcontext.py:1264-1269`) so retract/re-cross cannot re-farm them; `-0.05` wrong-commit is de-flickered by `WRONG_COMMIT_ARC_MM=10` depth gate.
- Success bonus factor 3.0 (`env5.py:313`) is consistent between reward and terminal.

2. Projection / geometry

BUG-G1 — Global nearest-point projection is not hairpin-safe → siphon arclength discontinuity

`eve/eve/util/polyline.py:82-144`, consumed via `PathProjectionCache.get_projection` (`pathcontext.py:504-520`)

`project_onto_polyline` takes the global `argmin` over all segments every step with **no continuity/monotonicity constraint and no "search near previous s" window**. At a hairpin (the carotid siphon in the LCCA path), the ascending and descending limbs are spatially adjacent; a tip near the bend is nearly equidistant to both limbs, so tiny tip jitter flips `best_idx` between a low-`s` segment and a high-`s` segment. The resulting Δs spikes by the full loop length in one step, which BUG-R1's doubling then converts into a large spurious progress reward (or a false backward penalty). - Direct tie to **siphon catheter-overshoot fold**: the reward/obs `d_rem`, tangent (features 2-3), and `curvature_ahead` all become erratic exactly where fine control is most needed. - Fix: constrain projection to a local arclength window around the previous `s` (e.g. ± 30 mm), or enforce monotone non-decreasing committed `s` with a bounded backward allowance. This is a geometry fix that does not touch the branch classifier.

INEFFICIENCY-G2 — O(B·N) per-branch projection scans repeated many times per step

- `get_nearest_branch` (`pathcontext.py:874-893`): projects onto every branch polyline. Feeds `is_on_correct_branch` (obs feature 7) each step.
- `classify_physical_branch` (`pathcontext.py:83-132`): full O(B·N) scan + per-branch `compute_cumulative_arclength` **rebuilt every call**. Called at minimum once/step for logging (`env5.py:1249`), plus conditionally at `env5.py:1010`, `1055`, `1417`, plus once in `MultiTargetEnv5.step` (`env5.py:2053`). Cumulative arclengths of static branches should be precomputed once at reset (they already are, in `_branch_cumlens`) and reused — `classify_physical_branch` ignores that cache and recomputes.
- Fix: cache `classify_physical_branch` result per step (it's a pure function of the tip); reuse `_branch_cumlens`.

Geometry checked and correct

- `compute_cumulative_arclength`, `compute_segment_tangents`, `compute_curvature` are correctly vectorized with divide-by-zero floors.
- `_build_branch_index` / `_build_path_branch_sequence` / `_build_branching_graph` are reset-time only (correctly out of the hot path), with a single-point smoothing pass to suppress tiebreak flips (`pathcontext.py:747-750`).
- Dijkstra path build (`fixedpath.py`) runs once/episode; `step()` is a genuine no-op.

INEFFICIENCY-G3 — `PathProjectionCache.reset()` runs 2× (up to 4×) per episode

`LocalGuidance.reset` (`localguidance.py:254`) and `ArcLengthProgress.reset` (`arclengthprogress.py:76`) both unconditionally call `self._path_context.reset()`, which re-runs `_build_branch_index` (incl. the O(n_{branch}·n_{polyline}) on-path mask build, `pathcontext.py:583-592`), `_build_entry_points`, `_build_branching_graph` (Dijkstra), and `_build_path_branch_sequence`. On SOFA-restore episodes `env5.reset` re-invokes both resets again (`env5.py:649-655`) → 4×. Idempotent (no correctness bug) but wasteful. Fix: guard with a "path identity" check or reset the cache once from `env5.reset`.

3. is_on_correct_path / branch classification

CONCEPTUAL-C1 — Reward and obs read *two different* on-path classifiers

- `ArcLengthProgress` and obs feature 27 (`is_in_a_wrong_branch`) read the **state-machine** `is_on_correct_path` / `_on_planned_path` (`pathcontext.py:932-965`).
- Obs feature 7 (`on_correct_branch`) reads the **debounced nearest-branch** `is_on_correct_branch` (`pathcontext.py:895-930`, flip threshold 5).

These disagree by construction (different metric, different hysteresis). The agent receives two inconsistent "am I on the path?" bits in the same observation, and the reward keys on the one the obs feature 7 does *not* expose. Fix: expose the same signal the reward uses, or document/justify the split.

C2 — Standing rule ("never change the classifier to fix a reward") — mostly respected, with a caveat

`env5.py:1004-1014` (Fix 2) overrides a **local** `on_correct_path` variable (used only for `_off_branch_steps` /logging) when `classify_physical_branch == target`. It does **not** mutate `pc._on_planned_path` or the reward's `is_on_correct_path()`. So the classifier the heuristic reads is untouched — rule respected. **Caveat:** this creates a split-brain state — during an overshoot still-inside-the-target-daughter, `env5` treats the wire as on-path (no timeout) while `ArcLengthProgress` treats it as off-path (applies Δ off-arc shaping + `-0.007` tax). The reward and the termination logic disagree about the same physical state. Worth reconciling.

Classification checked and correct

- Junction commit uses one-sided 10 mm arclength hysteresis with a 20 mm dead-band (`pathcontext.py:1156-1170`) — no per-step flip-flop at forks.
- Radius-aware tolerance `max(2, 1.5·r)` clamped to `[2,12]` (`pathcontext.py:1040-1070`) prevents both jitter false-positives (wide trunk) and wedge false-negatives (narrow daughter).
- `_pick_off_path_branch` restricts to junction-local sister candidates (`pathcontext.py:1083-1123`), avoiding the old KD-tree flicker.

4. Observation (LocalGuidance, 30-dim)

BUG-01 — Branch-categorical + off-path features (19, 25-28) lag the projection features by one step

`localguidance.py:394-464` read `_on_planned_path`, `_current_branch_idx`, `get_off_path_arc_since_divergence()` — all products of `update_branch_state`, which runs **after** `observation.step()` (see BUG-R3). So within one obs vector, features 0-12 use the current projection while features 19/25/26/27/28 reflect the state machine as of the *end of the previous step*. At a fork the "current branch / is-in-wrong-branch" bits describe where the wire *was*, not where the projection now says it is. Internal inconsistency; fix is the same as BUG-R3 (advance the state machine before obs).

NIT-02 — Declared dimensionality and box bounds are stale/loose

- Class docstring and the task both say "28-dim"; the vector is actually **30-dim** (`localguidance.py:466-500`, `self.obs = np.zeros(30)`). Rename/relabel.
- `curvature_ahead / 10.0` (feature 5) is **not clipped**; at a sharp siphon vertex (angle $\approx\pi$ over ~ 1 mm, curvature ≈ 3) it stays <1 , but there is no guarantee it respects the declared `high=1.0`. Clip to `[0,1]`.

CONCEPTUAL-03 — No catheter-tip observability (catheter controlled semi-blind)

All pose features derive from `fluoroscopy.tracking3d[0..2]` = the **guidewire** DOF chain (`localguidance.py:295-306`, `_compute_bend_hat_2d`, `_compute_heading_error`). The catheter enters the observation only as `inserted_lengths[1]` (scalar) and `last_action`. There is **no catheter tip position, tangent, or cross-track feature**. The agent must control catheter advance from insertion length alone —

exactly the information deficit behind **catheter overshoot** at the siphon. `fluoroscopy.device_trackings3d` (per-device tracking) exists and is unused here. Fix: add catheter-tip cross-track / heading features (the "missing catheter-tip features" the checklist hypothesizes — confirmed absent).

O4 — Fork-disambiguation: obs is target-relative and mostly sufficient, but degenerate laterally

Features 8-10 (routed distance + direction to next correct daughter entry) and 2-3 (planned tangent) are computed per-grader from that grader's Dijkstra path, so two daughters that share the trunk get identical obs until their paths diverge and then correctly diverge — the info to pick the correct fork *is* present in principle.

However, everything is expressed as projection-onto-the-planned-path; when the tip is laterally wedged between two sister ostia the projection (and thus tangent/ `d_correct` /direction) is ambiguous, and feature 27 lags (BUG-O1). The documented "EXP-3 fork-disambiguating feature" is indeed **not implemented** — there is no explicit "which sister am I pointed at" signal independent of the planned-path projection. This plausibly underlies **LVA fork confusion ~40%** at the trunk-top (2)→(0)/(18) split, where lateral position is most ambiguous.

Feature 23-24 route-aware fork-count fix — checked correct

`localguidance.py:410-425`: denominator = `len(_path_branch_sequence_with_junctions)` (per-grader on-path junction count), and `_n_correct_commits` increments once per on-path junction crossing over the same list (`pathcontext.py:1156-1167,1270`). Numerator/denominator use the identical set, so `forks_correct_norm` maxes at 1.0 and scales correctly across LCCA/LVA/RCCA/RVA (which have different junction counts). Consistent. Minor: `forks_wrong_norm` can saturate because `_n_wrong_commits` is unlatched — cosmetic.

Observation checked and correct

- NaN/inf guards present throughout (`_compute_bend_hat_2d` , `_entry_direction` , `_compute_heading_error` all floor norms).
- Frame consistency: tangents/headings/bend are all rotated into tracking3d before 2D projection with zero image-center so translation cancels — consistent with `tracking3d_to_2d` .
- Tracking normalized per-episode (`NormalizeTracking2DEpisode`) then Memory-stacked ×2 (FILL) so velocity reads 0 at reset — correct.
- Target identity: `Target2D` uses the frozen per-grader coord in heatmap and the shared target in single-target (`target2d.py:50-65`) — no cross-mode leakage.

5. Termination / truncation

- **Terminated (success):** `dist(tip, target_coord3d) < threshold(=5 mm)` — `TargetReached` / `_CoordTargetReachedTerminal` (`terminal/targetreached.py` , `env5.py:166-181`). Single-target `info["success"]` reads the same shared target → consistent with the terminal.
- **Truncated:** `MaxSteps(600)` , `VesselEnd` , `SimError` (train), plus env5-level `fold_stall≥20` and `off_branch≥50` .

BUG-T1 — Success reachable without being inside the daughter

`CenterlineRandom` samples the target uniformly over **all** points of the daughter branch (`centerlinerandom.py:78-93`), including the most-proximal point at/near the ostium. Success is a 5 mm Euclidean ball around that point. Near a bifurcation, sister branches and the bridge are within ~5 mm, so a wire in the **bridge/sister** can satisfy `dist<5` for a proximally-sampled target without ever entering the daughter lumen. If the eval "Quality" metric requires genuine deep entry, terminal success is looser than Quality → success% over-counts. Fix: require the target to be sampled $\geq N$ mm distal into the daughter, and/or gate terminal on `classify_physical_branch == target`.

T2 — checked correct

`at_tree_end` (`vesseltree.py:87-111`) is endpoint+open-mouth based, sensible. `MaxSteps` counter reset per episode. The MDP-grounding stamp `truncated→terminated` (`env5.py:1422`) correctly makes failures absorbing so the critic stops bootstrapping — but see BUG-R2 for the missing penalty on the max_steps arm.

6. Action interface

CONCEPTUAL-A1 — Asymmetric translation limits make "neutral policy output" = +10 mm/s forward on BOTH devices

`dualdevicenav.py:164` sets `velocity_limit_low=[[-10,-1.5],[-10,-1.5]]`, high `=[30,1.5],[30,1.5]]`. `DualDeviceNav` is built with `normalize_action=False`, and the eve_rl runner maps policy `[-1,1]→[low,high]` (`single.py:113-116`): `action_translation = (a+1)/2*40 - 10`. So policy output `a=0` → **+10 mm/s insertion**, and holding position requires `a=-0.5`. A tanh-Gaussian policy initialized near 0 therefore **advances both the guidewire and the catheter at 10 mm/s by default** — a strong forward bias that must be unlearned. Rotation is symmetric (`a=0` → 0), so only translation is biased. - Tie-in: the default-forward catheter, combined with no catheter-tip obs (O3), is a plausible mechanism for **catheter overshoot** at the siphon. Fix: center the mapping (symmetric limits, or subtract the midpoint bias), or initialize the policy mean to the hold action.

A2 — Action masking checked, one asymmetry note

`monoplanestatic.py:82-130`: below-zero, max-length, and tree-end masks zero only the **translation** channel and are per-device — correct. `last_action` is stored as the *masked-and-clipped raw velocity*, then obs-normalized against the action space, so obs `last_action` correctly reflects executed (not commanded) motion. Note the reset value `last_action=0` normalizes to `-0.5` on the translation channel (consistent with A1), so the "no action" obs baseline is -0.5, not 0 — internally consistent but easy to misread.

7. Efficiency (summary of hot-path costs)

- **INEFFICIENCY-E1:** `is_info_step = True` every step (`env5.py:1099`) with `handler.flush()` every step (`env5.py:1275-1276`) — the comment itself notes "~14× more log lines." Each info step also builds long f-strings, calls `classify_physical_branch`, `get_routed_d_corr_to_next_daughter_entry`, multiple

`get_projection` /arclength queries, and flushes file+stderr. This is the dominant per-step overhead and should be gated (every-N or terminal) for training runs.

- **INEFFICIENCY-E2:** `classify_physical_branch` rebuilds per-branch cumulative arclengths every call and is called several times/step (G2).
- **INEFFICIENCY-E3:** double/quadruple `PathProjectionCache.reset()` per episode (G3).
- `eve.Env.step` deep-copies obs/reward/terminal/truncated/info every step (`env.py:75-81`) — inherent to eve, minor next to E1.

8. Coordinate consistency — checked correct

`vessel_cs_to_tracking3d` / `tracking3d_to_vessel_cs` are proper inverses (`coordtransform.py:27-62`, `rot` then `translate` vs `translate-back` then `rot-1`). All path features convert tip and tangents through the same rotation with zero image-center for direction vectors, so world-vs-device-local framing is consistent.

`tracking3d_to_2d` uniformly drops axis 1 everywhere (obs target, tangent 2D, bend). Insertion point is already in vessel-CS and used directly. No frame mismatches found.

Obs/reward design gaps that plausibly cap LCCA-siphon success at ~50-60% (reasoned from code)

1. **Front-loaded, unpunished-timeout reward structure (BUG-R2 + R4).** Junction commits (+3) and progress shaping (~+4, doubled) are banked on the *approach*; the +3 terminal is a minority of episode return; `max_steps` timeout costs nothing; over-threading the correct daughter costs -5 (VesselEnd) and overshoot -1. The expected-value-maximizing behavior after entering the daughter is to **stop and coast to timeout with reward banked**, not to risk the deep siphon push. This is the code-level generator of **shallow-stall-inside-daughter** and a hard ceiling well under 100%.
2. **Farmable in-daughter jitter (BUG-R1).** The 2× forward-only doubling gives net-positive reward for oscillating in place, actively reinforcing shallow dithering over depth.
3. **Hairpin-blind projection (BUG-G1).** The siphon is exactly where global nearest-point projection flips between limbs, injecting spurious $\pm\Delta s$ into progress, tangent, and curvature — the reward signal is noisiest precisely at the LCCA hairpin, so the gradient that should pull the tip through the siphon is corrupted.
4. **No catheter-tip observability + forward-biased action (O3 + A1).** The catheter is steered from a single insertion-length scalar with a default-advance action mapping. At the siphon, where the catheter must hold/retract while the guidewire leads, the agent lacks both the observation and the neutral action to do so → **catheter-overshoot fold**.
5. **Two disagreeing on-path signals + one-step-lagged branch features (C1 + O1 + R3).** At the moment of the fork decision the agent's "which branch / on-path" bits are stale and mutually inconsistent, and there is no explicit fork-disambiguation feature (O4) — consistent with fork-confusion failure modes.

Individually each is a 5-15% leak; together they compound to roughly the observed 50-60% plateau. The two highest-leverage, lowest-risk fixes suggested by the code: **penalize the max_steps timeout proportional to**

remaining `d_rem` (BUG-R2) and **window the polyline projection to a local arclength band** (BUG-G1) — neither touches the frozen branch classifier the heuristic depends on.

Key file:line index

- Farmable doubling: `eve/eve/reward/arclengthprogress.py:137-148`
- Unpenalized timeout: `training _scripts/util/env5.py:1073-1088`, `1422-1423`
- State-machine 1-step lag: `env5.py:816-817`, `872-878`; `pathcontext.py:959-965`; `arclengthprogress.py:107`
- Overshoot=-5 via VesselEnd: `env5.py:1051-1062`, `1074-1087`
- Hairpin projection: `eve/eve/util/polyline.py:82-144`; `pathcontext.py:504-520`
- Two classifiers: `pathcontext.py:895-930` vs `932-965`; obs `localguidance.py:332-337`, `452-454`
- Catheter obs gap: `localguidance.py:295-306`; `fluoroscopy/trackingonly.py:92-110` (unused per-device tracking)
- Action asymmetry: `dualdevicenav.py:164`; `monoplanestatic.py:82-93`; `eve_rl/eve_rl/agent/single.py:113-116`
- Success-without-entry: `intervention/target/target.py:23-29`; `centerlinerandom.py:78-93`; `env5.py:166-181`
- Per-step logging cost: `env5.py:1099`, `1249`, `1275-1276`

Deep Review: eve_rl Endovascular Navigation Stack

Agent: Final synthesis (main session) · Source file: 5.md

Method: I read the latest strategy docs, the SAC/AWAC core, the policy head, and the truncation/reward hot paths myself, and fanned out four parallel deep-review agents over the full doc history (17 files, ~12k lines), the `eve_rl` algorithm layer, the eve environment layer, and the training harness. Every finding below marked [verified] I confirmed against the code personally; the rest come from the agents with file:line evidence I reviewed.

1. The verdict

You are on the right track methodologically, and you're much closer to the goal than the eval numbers suggest — but the current plateau is not a hyperparameter problem. It's four or five identifiable defects in the MDP itself, plus two live infrastructure bugs. The pid19116 sweep already proved the deeper truth: the policy is ~86–97% capable when the MDP around it is fair. What's capping LCCA at ~50–60% is mostly written in the reward, observation, and action-interface code, not in AWAC's knobs.

The evolution shows genuinely strong engineering discipline — the inversion bug hunt (`RL_IMPROV_10` §11), per-state attribution, the "one variable per iteration" rule, and the functional-success reclassification are better experimental hygiene than most published RL work. But that same discipline has ossified into a constraint ("never touch reward/obs/terminal") that is now protecting bugs, not design decisions.

2. Bugs — Tier 1: shaping what the live run is learning right now

These five interact, and together they form a coherent code-level explanation of your three documented failure modes (shallow-stall, siphon overshoot, LVA confusion):

2.1 The in-daughter progress doubling is reward-farmable [verified]. `arclengthprogress.py:137-148`: forward motion in the target daughter earns $2 \times \text{pf} \Delta s$, but the matching backward motion is charged only $1 \times \text{pf} \Delta s$. Every forward-back oscillation cycle nets $+\text{pf} \Delta s$ for zero net progress, and a $\geq 0.5\text{mm}$ wiggle never trips the fold-stall detector. This is a direct positive incentive for shallow dithering inside the daughter — the exact failure you named "shallow-stall." The fix is to make depth potential-based: credit only new maximum arclength reached ($\text{pf} \cdot \max(0, s - \text{s_max_so_far})$), which is un-farmable by construction.

2.2 Timing out with banked reward is unpenalized — stalling is locally optimal [verified]. `env5.py:1073-1088`: the -5 failure penalty gates on vessel-end, fold-stall, or off-branch timeout. A pure `max_steps` truncation pays nothing — while the episode has already banked up to $+3$ in junction commits plus $\sim +4$ of (doubled) progress shaping. So "enter the daughter, dither, coast to timeout" ends around $+7$ with zero risk, while pushing for the siphon target risks vessel-end (-5 , see 2.3) for a $+3$ bonus. The expected-value-maximizing policy stops short of the target. Combined with 2.1, this is the single strongest structural reason success caps below 100%.

2.3 Overshooting the correct daughter is punished like entering a wrong one. A wire that threads correctly but runs to the vessel end trips `VesselEnd` $\rightarrow -5$ (the soft -1 overshoot path only fires in the off-branch-timeout branch, `env5.py:1051-1087`). Together, 2.1–2.3 say: approach is generously paid, stopping is free, and going deep is the only penalized action. Your reward currently teaches conservatism.

2.4 Neutral policy output = +10 mm/s forward on both devices [verified]. Translation limits are $[-10, +30]$ and the runner maps $[-1, 1]$ linearly (`single.py:113-116`), so action 0 — where a tanh-Gaussian initializes and where entropy regularization pulls the mean — commands +10 mm/s insertion on guidewire and catheter. Holding still requires outputting -0.5 . A built-in catheter-forward bias plus finding 2.5 is a clean mechanistic account of the siphon catheter-overshoot fold. Fix: center the mapping (or bias the policy-mean initialization to the hold action).

2.5 The catheter is controlled semi-blind. All pose features in the 30-dim LocalGuidance derive from the guidewire tracking; the catheter appears only as an inserted-length scalar and last-action. There is no catheter tip position, heading, or cross-track feature, even though fluoroscopy. `device_trackings3d` exists unused. The agent must infer catheter-tip state at the siphon — precisely where it must coordinate hold/retract against guidewire advance. This confirms your own EXP-3 hypothesis from the code side: the missing observability is real, and it's on the catheter, not just the fork.

2.6 All 16 workers explore an identical target sequence [verified mechanism]. `centerlinerandom.py:34` constructs `random.Random()` once in the parent process; workers receive deepcopies of that RNG state, and unseeded explore resets never reseed it (`centerlinerandom.py:40-42`). The Plan-v12 spawn reseed fixed the global RNGs only. Your effective explore-target diversity is $\sim 1/16$ of nominal and PER receives 16-way-correlated data — right now, in the live container. (Eval and heuristic seeding are unaffected: both pass explicit seeds.) One-line fix in the spawn reseed block.

3. Bugs — Tier 2: the AWAC learning machinery

3.1 The auto-entropy "anti-rail" mechanism self-disables [verified, my own derivation]. `target_entropy` = -4 (`sac.py:210`) but your healthy policy runs at entropy $\approx +2.65$. The alpha loss therefore pushes `log_alpha` down monotonically — alpha decays exponentially toward zero and the $+\alpha \cdot \log_{\pi}$ term in the AWAC loss (the "Plan v11 PRIMARY anti-rail mechanism," `sac.py:437-446`) vanishes early in training. It can only revive once entropy has already collapsed below -4 , i.e. after the policy is deeply railed. It is a lagging regulator with a huge dead zone — consistent with v2 still sliding 59%→43% late in the run. Fix: set `target_entropy` near the healthy operating point ($\approx +1$ to $+2$ for this action dim and std band), or better, regulate what you actually care about — drive alpha with a setpoint controller on `clamp_fraction` (you already log it).

3.2 Hard torch.clamp on `log_std` makes both rails sticky [verified]. `gaussianpolicy.py:44` : outside the band the gradient is exactly zero, so no loss term can pull a railed `log_std` back — only incidental trunk drift can. This is the actual mechanism of v1's one-way "ceiling explosion," and it means EXP-1 (`log_std_max` 0.0) works by saturation, not regulation — pre-clamp values pile up above the cap with no restoring force. The standard fix is the SAC-v2 soft parameterization: `log_std` = $\min + 0.5 \cdot (\max - \min) \cdot (\tanh(x) + 1)$.

3.3 The -20 log-prob floor zeroes the BC gradient on exactly the demos you need [verified].

`gaussianpolicy.py:76` : when the policy is confidently wrong (σ near the floor, mean far from a demo), the demo's log-prob falls below -20 , the clamp flattens it, and the gradient is exactly zero. In AWAC the weighted-BC term is the only improvement signal, so "confidently wrong at bif2" is a frozen state — the hook demos in your clean lane silently stop teaching. This plausibly explains both the v1 mean-never-locking-the-hook symptom and v2's long $\sim 25\%$ plateau before breakout. Fix: a leaky floor (e.g. $-20 + 0.05 \cdot (\log p + 20)$ below the knee, or $-\text{softplus-based}$) so direction survives with damped magnitude — and log the clamp-hit fraction.

3.4 Every network is one hidden layer, not two [verified]. `mlp.py:83-87` has no activation between the input layer and the first hidden layer — the author even left `# TODO: Add F.relu after input layer`. `Linear◦Linear` collapses to a single affine map, so your "256×256" policy and critics are effectively single-hidden-layer nets. Everything ever trained in the MLP era, including the 86% RCCA stack, ran at half the intended depth. The fix is one line but changes the function class — old checkpoints won't be comparable and can't be warm-started meaningfully.

3.5 Assorted, real but smaller: exploration noise is a scalar added to all 4 dims (perfectly correlated across devices) and unclipped beyond $[-1, 1]$ (`sac.py:235`); PER beta never reaches 1.0 (anneal denominator is explore steps, advance is per update — ends ~ 0.7 at 20M, ~ 0.43 at your 2M horizon); `reward_scaling` is plumbed everywhere and applied nowhere; the padding-mask path biases the alpha loss and AWAC diagnostics (episode-mode only); a first-update-after-warm-start uses stale $\alpha=1.0$; the SAC branch's non-detached alpha survives only by fragile `zero_grad` ordering.

4. Bugs — Tier 3: environment internals and eval integrity

State-machine one-step lag: `update_branch_state()` runs after reward and observation each step, and the `is_on_correct_path` cache isn't invalidated after the update — so the progress-vs-off-path switch, the -0.007 off-path tax, the off-branch counter, and obs features 19/25–28 all act on the previous step's classification while using the current projection. At fork-decision moments the agent's "which branch am I in" bits describe where it was, not where it is.

Hairpin-blind projection: `polyline.py:82-144` takes a global nearest-segment argmin with no continuity window. At the carotid siphon the ascending/descending limbs are adjacent, so tip jitter flips the projection between limbs, spiking Δs by the loop length — reward, `d_rem`, tangent, and curvature features are noisiest exactly where fine control matters most. Fix: window the search to $\pm 30\text{mm}$ of the previous arclength (pure geometry, doesn't touch the classifier).

Success without entry: targets sample uniformly over the whole daughter centerline including the ostium; success is a 5mm Euclidean ball, so a proximally-sampled target can be "reached" from the bridge or a sister branch. Quality mildly over-counts.

Eval integrity: a worker crash mid-eval nulls the seed lists before collection — the restarted worker plays one unseeded random episode and the "98-seed Quality" silently becomes a ~ 91 -episode number with noise mixed in. Separately, at $p \approx 0.5$ your 98-episode eval has $\pm 5\text{pp}$ standard error: the $43 \rightarrow 59 \rightarrow 43$ swing is $\sim 2-3\sigma$ and `best_checkpoint` argmax over that metric has winner's-curse bias — the "59.2%" is best read as "low-to-mid 50s."

The per-eval buffer dump: `_replay_save_interval=1` exports the entire replay buffer via uncompressed `np.savez` at every 250k eval. With the 11M buffer this grows toward $\sim 7\text{GB}$ per eval, doubles buffer-subprocess RAM during the `np.stack`, and blocks training synchronously. Calibrated for the RCCA 1M buffer, never revisited.

Latent traps: buffer save deadlocks any non-PER run at first eval (`export_all` unimplemented on uniform buffers); with any nonzero `embedder_layers` the shared embedder is stepped only by q1's optimizer while receiving gradients from all three losses; deterministic eval runs `forward_play` twice per step (corrupts hidden state if an LSTM ever returns); training resume never restores buffer/runner state and the update-budget formula stalls resumed runs.

5. Documentation and process findings

The docs are a genuine asset — but the trail has drifts that will bite future-you:

The "same stack that hit 86%" claim is false on three knobs: LCCA v1/v2 run pretrain 10000 (RCCA v3: 1000), `update_per_explore_step` 0.5 (v3: 1.0), buffer 11M (v3: 1M). When you attribute v1's collapse or benchmark v2 against RCCA, you're not comparing the same update:data ratio. Also `launch_lcca_awac_v1.sh`'s header is the RCCA v3 header verbatim.

Undocumented reversals: `grad_clip` 10.0 was rejected with a quantitative argument (`RL_IMPROV_9` §16), then 1.0 shipped with no recorded rationale. The wrong-commit penalty is -1 in one doc and "the original -0.05 " in the next with no bridge. Memory was removed as unnecessary and re-added as essential.

Metric drift: "success" has meant six different things across the doc series (positive reward, $R > 1.0$, strict TargetReached, reach%, clean-thread $\times 3$ definitions, functional $R \in [-1, 1]$) — cross-doc comparisons like "8% vs 1.1%" mix definitions. And `RL_IMPROV_10` §17 itself demonstrates strict Quality partially measures which truncation classifier fires first.

`RL_TRAINING_LIFECYCLE.md` is dangerously stale — it still describes the LSTM/episode-buffer/1-in-20-updates system. Anyone onboarding learns the wrong architecture. And Plans v11/v12 have no changes doc at all; their rationale lives only in launcher comments and my memory files.

6. Are you asking the right questions?

Mostly yes at the tactical level — the EXP-1/2/3 isolation frame is good science. But three framing corrections:

a) You're asking "which knob fixes the symptom" when the causes are now identifiable defects. "Does `balanced_fraction` 0.6 teach the hook?" is unanswerable while the BC gradient for hook demos is clamped to zero (3.3), the entropy regulator is inert (3.1), and the reward makes stopping short optimal (2.2). The knob experiments are measuring noise on top of a rigged game.

b) The standing rule needs a carve-out for bugs. "Never touch reward/obs/terminal without approval" correctly guards against thrash — but 2.1, the state-machine lag, and the projection flip are not design changes; they're defects in the implementation of the intended design. Treating them as frozen turns the rule into bug preservation. Keep the rule for design changes; exempt verified-defect fixes (with reward regression tests, below).

c) The deepest unasked question: why per-daughter policies at all? `RL_IMPROV_8` §35 argued for per-daughter-first for good local reasons, and §36's shared-model experiment was never run. Meanwhile the "HER ruled out" verdict (`RL_IMPROV_9` §14) rests on a conflation — see 7.2. The per-daughter frame multiplies every problem by four and throws away the shared trunk-navigation skill.

Also worth asking: is strict Quality the right objective at all? Your own §17 analysis shows functional success tracks real improvement better. Clinically, you also argued cervical-ICA capability is what matters. Consider promoting functional success (or a per-anatomical-band vector) to the primary metric and keeping strict as secondary.

7. Overlooked and alternative strategies — the "next level" roadmap

Ordered by expected leverage per unit of risk:

7.1 Fix the MDP first, and build the missing safety net (highest leverage, near-zero research risk)

Everything in Tier 1 (2.1–2.6) plus the projection window. But the durable upgrade is a reward property-test suite — the docs show five reward regimes in four generations, each breaking invariants the previous one established (the `list.add` latch bug alone silently ate the +1 signal for 2,544 episodes). Encode invariants as replay tests over golden episodes: no closed loop in state space nets positive reward; oscillation nets ≤ 0 ; success > any failure; ordering success $\gg$ partial $\gg$ dwell $\gg$ deep-wrong; terminal penalties fire on every truncation class. Run on every reward edit. This converts your recurring whack-a-mole into a regression wall.

Important consequence: fixing 2.1/2.2 changes the reward function $\rightarrow$ `lcca_awac_seed_v1.npz` rewards go stale. You already have the precedent and tooling (the `RL_IMPROV_9` §17 retro-patched caches): re-score the stored trajectories offline rather than re-harvesting from SOFA.

7.2 Revisit HER — the ruling-out was wrong, and it attacks your #1 bottleneck

`RL_IMPROV_9` rejected HER because "the reward is path-structured and per-episode state is unreconstructable." That conflates goal relabeling with reward reconstruction. The path-context state machine is a deterministic function of (tip trajectory, planned path) — and you already re-run rewards over stored trajectories offline. So: for every episode that ended in the "wrong" daughter, re-run the state machine offline with that daughter as the goal $\rightarrow$ a success demo for that goal. Your 38% LVA-confusion failures become an LVA success dataset for free. Sparse per-daughter successes are your single scarcest resource; HER is the data multiplier purpose-built for it. This pairs naturally with:

7.3 One goal-conditioned policy across daughters

Condition on (daughter one-hot, target depth/arclength) instead of four separate policies. The trunk skill, the arch, `bif2` — all shared; the hook and the fork commits become goal-dependent behavior of one network. This also makes the siphon problem a depth-conditioning problem you can curriculum over, and every daughter's data regularizes the others. Fine-tune per-daughter afterwards if a daughter lags. The infrastructure (per-target buffers, target-branch schedule, multi-target harvest from Plan v12) already exists.

7.4 Residual RL over your heuristic — you built the perfect substrate and aren't using it

You have per-daughter heuristics with ~92% LCCA reach. Instead of seeding a from-scratch policy with heuristic demos, let the policy output a bounded delta on the heuristic's action ($a = a_{\text{heur}} + \pi\theta(s)$, clipped). The exploration problem collapses from "learn to navigate from `z=345`" to "learn corrections where the heuristic fails" — the fork hook and the siphon. This is the standard trick in contact-rich robot manipulation for exactly this situation, and it composes with everything above. (One caution: the heuristic reads `is_on_correct_path`, so the standing classifier-freeze rule still applies.)

7.5 Modernize the critic instead of leaning on the grounding hack

truncated $\rightarrow$ terminated for time-limits was a pragmatic fix for critic divergence, but it teaches the critic that late-episode healthy states are worthless — and your own functional-success analysis proves many truncated states are recoverable near-successes. The modern replacements for "value divergence in off-policy continuous control" are cheap and well-validated: LayerNorm in the critics (single biggest stabilizer in recent literature — CrossQ, DroQ lineage), critic ensembles / DroQ-style dropout-ensembles to kill overestimation, and n-step (3–5) returns to propagate your dense shaping along 600-step episodes faster. With those in place,

restore correct time-limit bootstrapping for `max_steps` (keep terminated=True for genuinely stuck failure truncations — that part of the grounding is defensible), and add a time-remaining observation feature to resolve the aliasing properly.

7.6 Asymmetric actor-critic — you're sitting on privileged state and not using it

SOFA knows everything: full wire node positions, contact forces, catheter tip pose. Train the critic (and only the critic) on privileged state while the policy uses the deployable observation. Better value targets, no deployment cost. This directly attacks "critic quality," which your entire stabilization saga has been fighting with blunter tools. Natural extension: a teacher policy trained with privileged obs + restore-anywhere curriculum, distilled into the deployable-obs student (this is how most sim-trained navigation/locomotion results get their final performance).

7.7 Curriculum, formalized — you already proved it works (+53pp) and then didn't systematize it

Restore-at-fork was the single most effective intervention in the whole history. Generalize it: reverse curriculum from the target backwards (start states sampled progressively further from the goal as success rises), and a bandit over start-states/target-bands driven by your per-state success tables (sample where learning progress is highest, not uniformly over a hard-skewed distribution). The siphon-band bias you proposed in §19.4 is one cell of this; do the whole grid automatically.

7.8 Simplification candidates (subtract, don't add)

The sampler is currently $PER \times \text{balanced-two-stream} \times \text{demo-bonus}$ — a triple-biased estimator whose IS correction is provably not converging to unbiased (agent finding D4) and was never A/B'd against uniform. Given AWAC's own weighting already does prioritization-by-advantage, try uniform sampling + the clean lane as the ablation before tuning PER further. Similarly, the 30-dim guidance vector has never had a feature ablation at any size (8→14→28→30); with two of its features provably stale-lagged and one heuristic-phase feature that's semantically undefined during RL exploration, an ablation would likely both simplify and improve.

7.9 Bigger swings, if you want research upside

Wire dynamics are hysteretic (contact state, friction, stored bending energy) — a genuine POMDP. Options in ascending ambition: longer frame-stack on tip history; a small GRU (now safe post-step-RL, if J3's double-forward is fixed); a learned dynamics model for short-horizon MPC or Dreamer-style imagination — at ~6 explore steps/s aggregate, your sim is expensive enough that model-based amplification has real ROI; distributional critics (QR-DQN-style) for the heavy-tailed outcome structure. These are "after the fundamentals" items.

8. What I'd actually do, in order

Today, on the live run (no restart needed): decide whether to let it run — evidence says it's plateaued in a 43–59% band with a self-disabled entropy regulator and 1/16 target diversity, so its marginal value is checkpoint archaeology, not learning. Also note: your repo files are bind-mounted into the container, so any host-side edits will silently go live on a container restart — sequence your edits knowing that.

"v3" restart package (bug fixes only, no design changes): target-RNG per-worker reseed;

`_replay_save_interval` gate; eval-count logging next to Quality; MLP ReLU; soft `log_std` parameterization; leaky log-prob floor; `target_entropy` recalibration (or clamp-fraction-driven alpha); noise size=+clip; PER beta denominator. Re-score the seed cache offline after the reward fixes (2.1, 2.2, 2.3) go in. Expect discontinuity vs. all prior numbers — that's the point.

First design experiment after v3 stabilizes: catheter-tip observation features + centered action mapping (these two directly target the siphon mechanism and are the code-confirmed version of your EXP-3 instinct — stronger than the fork-feature variant).

The strategic fork: goal-conditioned single policy + offline HER relabeling of the existing multi-daughter harvest, warm-started via IQL (your Plan-v12 scaffold) — with residual-over-heuristic as the fallback if from-obs learning still struggles.

Process: reward property tests; promote functional success to primary metric with CIs; one-paragraph Plan v11/v12 changes doc; retire or rewrite `RL_TRAINING_LIFECYCLE.md`.

The honest summary: the last four months built an excellent diagnostic culture and a nearly-capable policy, and then spent increasing effort tuning stabilization knobs around defects that were invisible without a line-level audit. Fix the six Tier-1 items and recalibrate the entropy mechanism, and I'd expect the existing AWAC stack — before any of the fancier strategies — to move LCCA well past the current band, because you'll finally be training against a reward that actually wants what you want.